

Министерство цифрового развития, связи и массовых коммуникаций РФ
Уральский технический институт связи и информатики (филиал)
ФГБОУ ВО «Сибирский государственный университет
телекоммуникаций и информатики» в г. Екатеринбурге
(УрТИСИ СибГУТИ)

Отчёт
по учебной практике
по МДК 02.02 «Инструментальные средства разработки
программного обеспечения»
на тему «Разработка приложения по варианту ДЭ
и реализация дипломного проекта»

студента _____ IV _____ курса _____ 285 _____ группы
Фамилия _____ Машковцев _____
Имя, отчество _____ Сергей Александрович _____
Факультет _____ Инфокоммуникаций, информатики и управления _____
Специальность _____ 09.02.07 «Информационные системы
и программирование» _____

Екатеринбург, 2026

Отзыв руководителя

Содержание

Введение	4
1 Демонстрационный экзамен	5
1.1 Анализ предметной области и технического задания	5
1.2 Выбор стека технологий и инструментов разработки	5
1.3 Работа с базой данных	6
1.4 Практическая реализация решения	8
1.5 Интерфейс программы	11
1.6 Тестирование программы	13
1.7 Качественные характеристики кода	14
1.8 Предложения модификации программы	15
1.9 Руководство системному программисту	16
2 Дипломный проект	20
2.1 Введение по дипломному проекту	20
2.2 Анализ предметной области	21
2.3 Требования к продукту	22
2.4 Обоснование выбора инструментальных средств разработки	24
2.5 Разработка структуры веб-приложения	29
2.6 Проектирование и разработка базы данных	30
2.7 Разработка пользовательского интерфейса	32
2.8 Доработка приложения	35
2.9 Разработка дополнительного функционала	39
Заключение	45
Библиография	46
Приложение А	47
Приложение Б	49

					09.02.07.000022 П.515			
Изм.	Лист	№ докум.	Подпись	Дата				
Разраб.		Машковцев С.А.			Отчёт по МДК 02.02 «Инструментальные средства разработки программного обеспечения»	Лит.	Лист	Листов
Пров.		Бурумбаев Д.И.					3	46
Реценз.						УрТИСИ СибГУТИ		
Н. контр.								
Утв.								

Введение

Данная учебная практика направлена на подготовку к демонстрационному экзамену и описанию практической части дипломного проекта.

Цель учебной практики – проработать вариант демонстрационного экзамена и составить пояснительную записку к практической части дипломного проекта.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать техническое задание и предметную область;
- сформировать базу данных и архитектуру программы;
- разработать приложение;
- разработать документацию к продукту;
- протестировать программу;
- проработать приложение дипломного проекта.

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		4

1 Демонстрационный экзамен

1.1 Анализ предметной области и технического задания

Разрабатываемый модуль предназначен для автоматизации процессов регистрации, обработки и мониторинга заявок на ремонт климатического оборудования в сервисном центре "IT-Сфера".

Согласно техническому заданию, программа должна соответствовать следующим функциональным требованиям:

- 1) возможность добавления заявок в базу данных;
- 2) возможность редактирования заявок;
- 3) возможность отслеживания статуса заявки;
- 4) возможность назначения ответственных за выполнение работ;
- 5) расчет статистики работы отдела обслуживания.

Также программа должна соответствовать следующим нефункциональным требованиям:

- 1) кроссплатформенность;
- 2) безопасность;
- 3) удобство использования;
- 4) производительность.

Из конечных требований хочется отметить, что язык программирования и система управления баз данных могут быть выбраны на усмотрение разработчика.

Входные данные:

- 1) учетные данные пользователей (логин, пароль);
- 2) данные от заказчиков (тип оборудования, модель, описание проблемы);
- 3) данные от специалистов (комментарии, запчасти, смена статуса).

Выходные данные:

- 1) списки заявок с фильтрацией по статусам, клиентам или специалистам;
- 2) уведомления системы об ошибках или успешных действиях;
- 3) статистические отчеты (количество выполненных заявок, среднее время выполнения, статистика неисправностей);
- 4) сгенерированный QR-код для оценки качества.

1.2 Выбор стека технологий и инструментов разработки

Для реализации программного обеспечения был выбран стек технологий на платформе .NET с использованием языка программирования C#. В качестве основной среды разработки использовалась интегрированная среда Visual Studio, обеспечивающая удобные средства редактирования кода и отладки. Язык C# и платформа .NET были выбраны благодаря наличию большого количества библиотек и стабильной работе в среде Windows.

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		5

Для разработки веб-интерфейса применён фреймворк ASP.NET Core MVC. Он реализует шаблон Model-View-Controller, что позволяет разделить ответственность между слоями представления, бизнес-логики и доступа к данным. Данный фреймворк упростил обработку форм, работу с моделями и представлениями.

В качестве системы управления базами данных была выбрана СУБД PostgreSQL. Она обеспечивает надёжность хранения данных и поддержку транзакций. Для администрирования и резервного копирования базы данных использовался инструмент pgAdmin 4, позволяющий выполнять операции создания и изменения структуры базы данных, управления таблицами и выполнения резервного копирования в графическом интерфейсе.

Доступ к данным реализован с помощью технологии Entity Framework Core. Она позволяет работать с данными на уровне объектов C#, не формируя вручную SQL-запросы. EF Core обеспечивает привязку сущностей к таблицам базы данных, автоматическое формирование запросов.

Для реализации механизмов аутентификации и авторизации использовались средства ASP.NET Core, основанные на применении cookie-аутентификации и наборе атрибутов пользователя. При входе пользователя в систему формируется набор (ФИО, роль, идентификатор пользователя), который далее используется фреймворком для проверки прав доступа через атрибуты [Authorize] и методы User.IsInRole. Такой подход позволяет управлять правами пользователей, разграничивать доступ к разделам приложения и обеспечивать безопасную работу с данными заявок и комментариев.

1.3 Работа с базой данных

Перед созданием базы данных были проанализированы файлы с данными и были выделены следующие сущности и их атрибуты:

- 1) roles (роли):
 - roleID (PK);
 - roleName;
- 2) statuses (статусы):
 - statusID (PK);
 - statusName;
- 3) users (пользователи):
 - userID (PK);
 - fio;
 - phone;
 - login;
 - password;
 - roleID (FK);
- 4) requests (заявки):

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		6

- requestID (PK);
- startDate;
- climateTechType;
- climateTechModel;
- problemDescription;
- statusID (FK);
- completionDate;
- repairParts;
- masterID (FK);
- clientID (FK);
- 5) comments (комментарии):
- commentID (PK);
- message;
- masterID (FK);
- requestID (FK).

Связи между сущностями:

- роли и пользователи (один ко многим): одна роль может быть у нескольких пользователей;
- статусы и заявки (один ко многим): один статус может быть у множества заявок;
- пользователи и мастера (один ко многим): один мастер может выполнять множество заявок;
- пользователи и клиенты (один ко многим): один клиент может оставить множество заявок;
- заявки и комментарии (один ко многим): одна заявка может иметь несколько комментариев.

ERD-диаграмма системы представлена на рисунке 1.1.

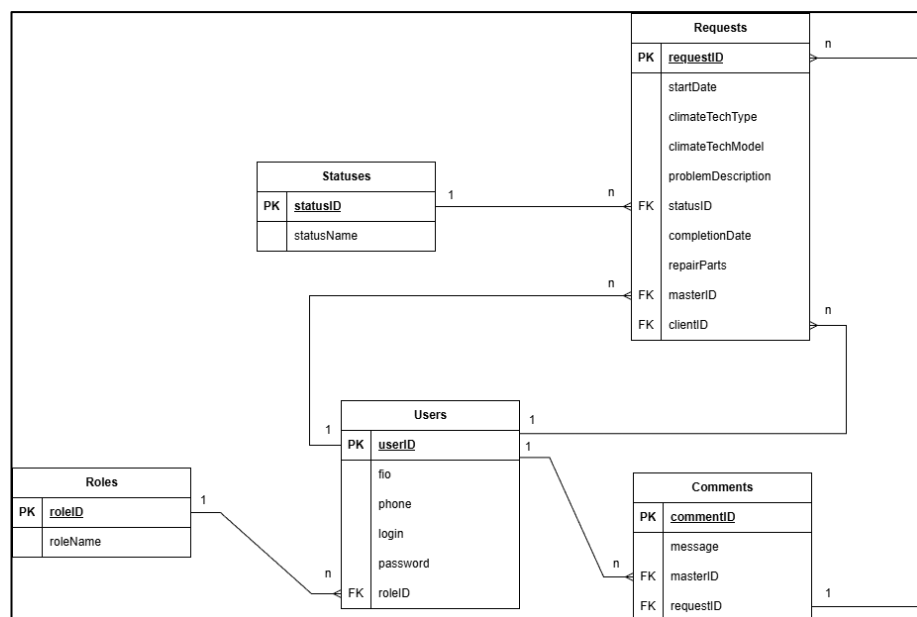


Рисунок 1.1 – ERD-диаграмма

Запросы создания и заполнения базы данных представлены в приложении А.

1.4 Практическая реализация решения

Перед созданием программы, согласно заданию, было сформировано несколько блок-схем по функционалу программного обеспечения. Блок-схема основного алгоритма решения учёта заявок на ремонт климатического оборудования представлена на рисунке 1.2. Данный алгоритм показывает последовательность действий пользователя и выполнение проверок прав доступа в зависимости от роли.

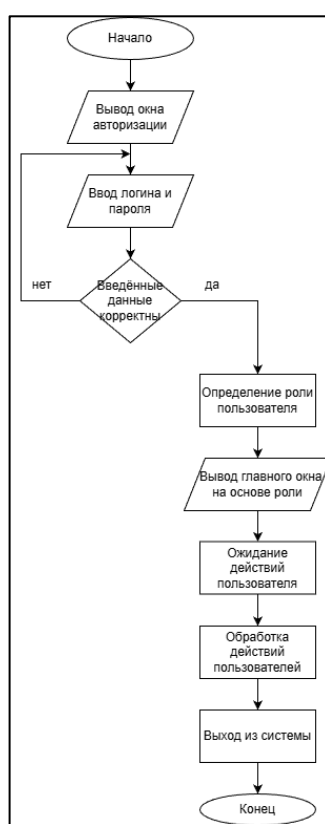


Рисунок 1.2 – Основной алгоритм работы системы

Блок-схема алгоритма расчёта количества заявок представлена на рисунке 1.3.

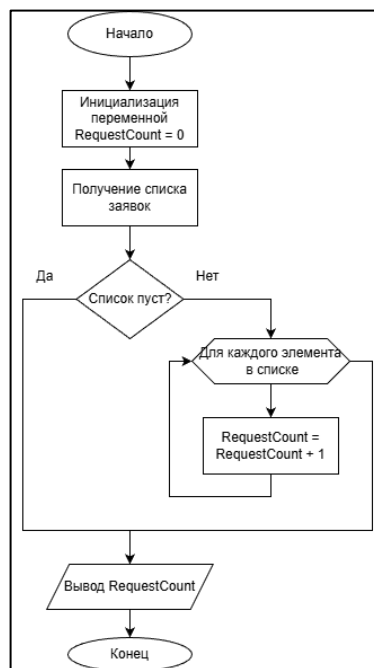


Рисунок 1.3 – Блок-схема расчёта количества заявок

Программное обеспечение реализовано в виде веб-приложения на основе шаблона MVC, что позволяет разделять ответственности между компонентами системы.

Компоненты системы:

- Model: сущности базы данных реализованы классами User, Role, Request, Status, Comment. Для привязки сущностей к таблицам применены атрибуты [Table("...")]. Между сущностями есть связи: заявка связана со статусом, заказчиком и специалистом; комментарий связан с заявкой и специалистом;

- Data: для работы с БД используется ApplicationDbContext, он содержит наборы сущностей (DbSet) и конфигурирует связи через OnModelCreating, для предотвращения потери связанных данных установлено ограничение удаления DeleteBehavior.Restrict (то есть, нельзя удалить пользователя, если существуют связь с какой-либо заявкой);

- View: пользовательский интерфейс реализован Razor-представлениями. Данные в представления передаются через модель (View(model)) и дополнительные параметры, например для формирования выпадающих списков статусов, клиентов и специалистов;

- Controller: основная логика работы с заявками реализована в RequestsController. Также есть и другие контроллеры, такие как AccountController (вход и выход пользователя, формирование пользовательской сессии), CommentsController (работа с комментариями), StatisticsController (расчёт и вывод статистики) и HomeController (навигация);

- разделение по ролям: методы IsOperatorOrManager, IsSpecialist, IsClient определяют права, также во всех действиях контроллера проверяется роль и владение заявкой;

– действия по ролям: разные поля доступны для изменения разным ролям (клиент может менять описание, специалист — статус, детали ремонта и даты, оператор/менеджер — всё, включая назначение мастера и клиента), также статусы заявок берутся из отдельной таблицы `Statuses`.

Вход пользователя реализован в `AccountController`. При успешной проверке логина и пароля из БД формируется набор пользовательских атрибутов (ФИО, роль, идентификатор пользователя), который сохраняется в `cookie`-сессии. Далее во всех контроллерах доступ к разделам ограничивается атрибутом `[Authorize]`, а права уточняются по роли и идентификатору пользователя.

Работа с заявками реализована с помощью контроллера `RequestsController`, содержащего набор операций:

– список заявок: формируется запрос к `_context.Requests` с подгрузкой связанных сущностей (`Status`, `Client`, `Master`), также реализованы фильтры по роли и по номеру, статусу и тексту заявки, заявки сортируются по дате создания;

– карточка заявки: загружается заявка по `id` со связями, выполняется проверка права просмотра, также подгружается список комментариев к заявке и вычисляется флаг возможности добавления комментария (только специалист, назначенный на заявку);

– создание заявки: перед отображением формы выполняется проверка роли, загружаются статусы, а для оператора/менеджера также список клиентов, при создании по умолчанию устанавливается дата и статус «Новая заявка», а при сохранении выполняются проверки корректности введенных данных, также происходит приведение к единому формату времени;

– редактирование заявки: при редактировании заявка загружается из БД по `id`, далее разрешенные поля зависят от роли (заказчик может изменять описание проблемы, специалист – статус, детали ремонта, дату выполнения, оператор или менеджер – все данные, включая назначение специалиста и изменение статуса);

– удаление заявки: удаление доступно только оператору/менеджеру, при невозможности удаления пользователю выводится сообщение о причине.

`CommentsController` обеспечивает добавление комментариев к заявкам специалистом:

– добавление комментария допускается только если специалист назначен на соответствующую заявку;

– есть проверка пустого сообщения и обработка типовых ошибок;

– администрирование ограничено ролями оператора и менеджера.

`StatisticsController` формирует статистику по заявкам:

– общее количество завершенных заявок;

– есть проверка пустого сообщения и обработка типовых ошибок;

– группировки по типу техники и статусам, а также наиболее частые проблемы.

Для повышения удобства использования реализованы обработки ошибок и сообщений:

– проверки `NotFound`, `Forbid`, `Challenge` для корректной обработки отсутствующих данных и нарушений доступа;

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		10

- обработка исключений при сохранении в БД (DbUpdateException, общий catch) с выводом понятного сообщения;
- вывод информационных сообщений пользователю через TempData["Info"] (создание заявки, изменение статуса, добавление комментария и другие).

1.5 Интерфейс программы

В процессе разработки программы было реализовано следующее:

- меню вкладок с таблицами;
- права доступа согласно ролям;
- авторизация по логину и паролю;
- CRUD операции;
- обработка ошибок и предупреждений при удалении данных;
- вкладка со статистикой работы отдела;
- отображение данных текущего пользователя и функция выхода из системы.

Интерфейс программы представлен на рисунках 1.4-1.10.

Рисунок 1.4 – Окно авторизации

ID	Оборудование	Проблема	Клиент	Мастер	Статус	Дата	Действия
5	Сушилка для рук (Ballu BAHND-12501)	Не работает	Кузнецов Сергей Матвеевич	Кудрявцева Ева Ивановна	Новая заявка	02.08.2023	Открыть Изменить Удалить
4	Увлажнитель воздуха (Polaris PUH 2300 WIFI IQ Home)	Продолжает работать при снижении воды	Ковалева Софья Владимировна	Беспалова Екатерина Данильевна	Новая заявка	02.08.2023	Открыть Изменить Удалить
1	Кондиционер (TCL TAC-12CHSA/TPG-W белый)	Не охлаждает воздух	Петров Никита Артёмович	Кудрявцева Ева Ивановна	В процессе ремонта	06.06.2023	Открыть Изменить Удалить
2	Кондиционер (Electrolux EACS/I-09NAT/N3_21Y белый)	Выключается сам по себе	Ковалева Софья Владимировна	Гончарова Ульяна Ярославовна	В процессе ремонта	05.05.2023	Открыть Изменить Удалить
3	Увлажнитель воздуха (Xiaomi Smart Humidifier 22)	Пар имеет неприятный запах	Кузнецов Сергей Матвеевич	Гончарова Ульяна Ярославовна	Готова к выдаче	07.07.2022	Открыть Изменить Удалить

Рисунок 1.5 – Главная вкладка

Заявки

Статистика

Комментарии

Широков Василий Матвеевич

Менеджер

Выход

Статистика

Выполнено заявок: 1

По статусам

Статус	Кол-во
В процессе ремонта	2
Новая заявка	2
Готова к выдаче	1

По типам оборудования

Тип	Кол-во
Кондиционер	2
Увлажнитель воздуха	2
Сушилка для рук	1

Топ-10 проблем (как в тексте заявки)

Проблема	Кол-во
Выключается сам по себе	1
Не охлаждает воздух	1
Пар имеет неприятный запах	1
Продолжает работать при снижении воды	1
Не работает	1

Рисунок 1.6 – Окно статистики

Заявки

Статистика

Комментарии

Широков Василий Матвеевич

Менеджер

Выход

История комментариев

ID	Сообщение	Мастер	Заявка №2
3	Починим в момент.	Гончарова Ульяна Ярославовна	3
2	Всё сделаем!	Гончарова Ульяна Ярославовна	2
1	Всё сделаем!	Кудрявцева Ева Ивановна	1

Рисунок 1.7 – Окно комментариев

Заявки

Статистика

Комментарии

Широков Василий Матвеевич

Менеджер

Выход

Создать заявку

Дата добавления

04.03.2026

Тип оборудования

Модель

Описание проблемы

Статус

Новая заявка

Клиент

Ковалева Софья Владимировна

Сохранить

Назад

Рисунок 1.8 – Создание заявки

Редактировать заявку №4

Дата добавления

02.08.2023

Тип оборудования

Увлажнитель воздуха

Модель

Polaris PUH 2300 WIFI IQ Home

Описание проблемы

Продолжает работать при снижении воды

Статус

Новая заявка

Мастер

Беспалова Екатерина Данильевна

Комплектующие

Дата завершения

дд.мм.гггг

Сохранить

Назад

Рисунок 1.9 – Редактирование заявки

Заявки

Петров Никита Артёмович

Заказчик

Выход

Заявки

№ заявки

Любой статус

Текст (оборудование/модель/проблема/клиент)

Поиск

Сброс

+ Добавить

ID	Оборудование	Проблема	Клиент	Мастер	Статус	Дата	Действия
1	Кондиционер (TCL TAC-12CHSA/TPG-W белый)	Не охлаждает воздух	Петров Никита Артёмович	Кудрявцева Ева Ивановна	В процессе ремонта	06.06.2023	Открыть Изменить

Рисунок 1.10 – Главная страница с роли заказчика

1.6 Тестирование программы

После реализации программы она была протестирована, тесты и их результаты представлены в таблице 1.

Таблица 1 – Тестирование программы

№ теста	Модуль тестирования	Порядок тестирования	Ожидаемый результат	Результат теста
1	Авторизация	Ввод корректного пароля и логина	Вход в систему	Успешный вход
2	Авторизация	Ввод неправильного пароля или логина	Появление ошибки	Вывод ошибки без входа в систему

3	Ролевой доступ	Поочерёдный вход в систему под разными ролями пользователей	Успешное отображение таблиц в соответствии с ролями	У каждой роли отображается свой функционал
4	Работа с данными	Создать заявку	Заявка успешно создана	Успешно созданная заявка
5	Работа с данными	Удалить заявку	Заявка удалена	Успешно удаленная заявка
6	Работа с данными	Изменение заявки	Заявка изменена	Успешно измененная заявка
7	Работа с данными	Добавление комментария	Появление комментария	Не созданный комментарий к заявке
8	Работа с данными	Создание заявки с пустыми полями		Отображение ошибки о некорректности введенных данных

Согласно тестированию, все тесты за исключением одного были успешно выполнены. Тест прошёл неудачно в модуле работе с данными, а именно не был создан комментарий.

1.7 Качественные характеристики кода

После тестирования происходила оценка кода. Определение качественных характеристик кода:

- 1) полнота обработки ошибочных данных:
 - присутствуют: проверки на null/отсутствие сущностей, int.TryParse для UserId, проверка пустых строк, ветки по ролям, try/catch вокруг SaveChangesAsync и DbUpdateException с выводом понятного сообщения;
 - отсутствуют: нет централизованного логирования ошибок, нет разборчивой обработки разных типов ошибок БД, ограничения по длине, форматам и т.д. минимальны;
 - итог: желательно дополнить;
- 2) наличие тестов для проверки допустимых значений входных данных:
 - тесты отсутствуют;
- 3) наличие средств контроля корректности входных данных:
 - присутствуют: использование ModelState.IsValid, Bind(...) для ограничения привязки, ручные проверки (пустой текст комментария, выбран ли клиент,

конвертация дат в UTC), проверки принадлежности заявки пользователю по ClientID/MasterID;

- отсутствуют: атрибуты валидации на моделях есть не везде;

- итог: базовый контроль есть, но можно доработать;

4) наличие средств восстановления при сбоях оборудования:

- отсутствуют: в приложении не реализованы автоматическое резервное копирование, повторные попытки, транзакции с откатом;

- итог: средств восстановления нету;

5) наличие комментариев:

- присутствуют: краткие комментарии над методами и полями, поясняющие назначение основных действий;

- отсутствуют: нету ненужных и больших по количеству комментариев;

- итог: комментарии присутствуют в достаточной мере и описывают назначения основных компонентов;

6) наличие проверки корректности передаваемых данных:

- присутствуют: проверка параметров маршрута на существование сущностей в БД, проверки прав доступа по ID пользователя, проверка того, что специалист добавляет комментарий только к своей заявке;

- итог: проверка корректности передаваемых данных присутствует;

7) наличие описаний основных функций:

- присутствуют: основные функции имеют понятные имена (create, edit, delete, index) и краткие комментарии «что делает функция» (создание, редактирование, удаление, расчёт статистики, просмотр комментариев и остальные);

- итог: описание основных функций присутствует на уровне кода (имена и краткие комментарии).

1.8 Предложения модификации программы

Разработанное ПО можно дополнить следующим новым функционалом:

1) работа с пользователями (редактирование, удаление, блокировка учетных записей, просмотр истории входов пользователей);

2) добавить сроки выполнения заявки и уведомления об несвоевременном выполнении заявки или её запоздании;

3) добавление файлов при оформлении заявки (фото или видео поломки);

4) более расширенная система отчётности – статистика по работе (по определенному мастеру или клиенту), различные диаграммы;

5) система уведомлений;

6) расширить возможности поиска и фильтров (по дате, по типу техники и длительности ремонта);

7) применение стилей для «красоты» интерфейса;

8) возможность оставления отзывов клиентами по выполненным заявкам.

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		15

1.9 Руководство системному программисту

1.9.1 Общие сведения о программе

Данное руководство предназначено для изучения программы под названием «Программный модуль для учета заявок на ремонт климатического оборудования». Программа предназначена для ведения жизненного цикла заявок на ремонт климатической техники: создание, просмотр, фильтрация, назначение мастера, изменение статусов, фиксация результата ремонта и ведение комментариев специалистов. Документ содержит сведения, необходимые системному программисту для развертывания, настройки, проверки работоспособности и сопровождения веб-приложения. Описаны структура программы, порядок настройки параметров окружения и базы данных, способы проверки, дополнительные возможности и сообщения, возникающие при работе программы и требующие действий.

Основные функции программы:

- список заявок с фильтрами, карточка заявки, создание, редактирование и удаление (с разграничением по ролям);
- поддержка ролей пользователей (менеджер, оператор, специалист, заказчик) и ограничение доступа к функциям;
- добавление комментариев специалистом к назначенной ему заявке, а также просмотр комментариев;
- расчёт показателей по заявкам (для оператора/менеджера).

Технические средства:

- 1) компьютер с операционной системой Windows 10/11 или Linux, поддерживающий .NET;
- 2) СУБД PostgreSQL (локально или на сервере);
- 3) сеть или локальный доступ к БД.

Программные средства:

- 1) .NET / ASP.NET Core (MVC);
- 2) Entity Framework Core + Npgsql (PostgreSQL);
- 3) Cookie-аутентификация ASP.NET Core;
- 4) веб-браузер для работы пользователя.

1.9.2 Структура программы

Структура программы состоит из нескольких частей:

- 1) точка входа "Program.cs" (регистрация MVC, EF Core контекста, аутентификации Cookie, также маршрутизация и инициализация БД);
- 2) доступ к данным "ApplicationDbContext.cs" (DbSet: Users, Roles, Statuses, Requests, Comments, также настройка связей и ограничений удаления);

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		16

- 3) модели данных (User, Role, Status, Request, Comment);
- 4) контроллеры (AccountController – вход/выход, RequestsController – заявки, CommentsController – комментарии, StatisticsController – статистика, HomeController – переадресация, разделы);
- 5) представления (формы, таблицы, страницы карточек, общий макет _Layout, верхнее меню _TabMenu).

Связи между частями программы:

- 1) контроллеры получают ApplicationDbContext через внедрение зависимостей и выполняют операции CRUD и выборки запросов;
- 2) представления получают модели или данные из ViewBag (списки статусов, пользователей, статистика);
- 3) система авторизации выдаёт информацию (ФИО, роль, ID пользователя), которые используются контроллерами для проверки прав и фильтрации данных.

Связи с другими программами:

- 1) PostgreSQL (внешний сервис хранения данных);
- 2) веб-сервер Kestrel (встроен в ASP.NET Core).

1.9.3 Настройка программы

Настройка окружения:

- 1) установить .NET 10 версии;
- 2) установить и запустить PostgreSQL;
- 3) настроить сетевую доступность PostgreSQL для приложения (host, port, user, password).

Настройка подключения к базе данных:

- 1) в конфигурации приложения должна быть задана строка подключения DefaultConnection (используется в Program.cs);
- 2) в качестве провайдера выступает Npgsql, также включено соглашение нижнего регистра поэтому имена таблиц и полей ожидаются в малой форме букв.

Инициализация базы данных:

- 1) при старте выполняется:
 - EnsureCreated() – создание таблиц при отсутствии;
 - добавление ролей и статусов при пустых таблицах;
- 2) системный программист должен контролировать права пользователя БД на создание таблиц и выполнение SQL.

Настройка ролей и пользователей:

- роли создаются автоматически: менеджер, специалист, оператор и заказчик;
- пользователи должны быть заведены в таблицу Users;

– приложение использует проверку логина и пароля прямым сравнением с БД (системному программисту важно обеспечить корректность исходных данных и доступ к БД).

Настройка функций по ролям:

– заказчик (создание заявки, просмотр и редактирование своих данных заявки);

– специалист (работа с назначенными заявками, изменение статуса и результатов ремонта, комментарии по своим заявкам);

– оператор и менеджер (более обширное управление заявками (назначение мастеров, удаление), доступ к статистике, администрирование комментариев).

1.9.4 Проверка программы

Проверка программы проводится через веб-интерфейс в браузере с использованием тестовых пользователей разных ролей.

Список проверок:

1) проверка входа:

– ввести корректный логин и пароль (успешный вход);

– ввести некорректный логин и пароль (неуспешный вход);

2) проверка заявок у заказчика:

– создать заявку и получить сообщение, что заявка создана;

– убедиться, что видны только свои заявки;

3) проверка заявок у специалиста:

– открыть доступную заявку (данные отображаются);

– изменить данные заявки и получить сообщение об успешном изменении;

– добавить комментарий (комментарий добавлен);

4) проверка заявок у оператора:

– создать заявку (заявка создана);

– назначить мастера (мастер успешно выбран);

– удалить заявку (заявка удалена);

5) проверка статистики:

– открыть вкладку статистики под оператором (страница видна);

– зайти на вкладку с других ролей (вкладка закрыта).

1.9.5 Дополнительные возможности

Дополнительные возможности:

1) статистика:

– количество завершённых заявок;

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		18

- распределения по типу техники, статусу, наиболее частым проблемам (топ-10);
- доступ только у оператора и менеджера;
- 2) администрирование комментариев:
 - редактирование и удаление комментариев оператором/менеджером;
- 3) просмотр неиспользуемых таблиц для обычных пользователей (Users, Roles, Statuses):
 - просмотр списков пользователей/ролей/статусов (раздел для администрирования).

1.9.6 Сообщения системному программисту

Программой могут быть выданы следующие сообщения:

- 1) «Неверный логин и пароль»:
 - означает, что данные пользователя введены некорректно;
 - действия: проверить введенные данные, проверить наличие пользователя;
- 2) «Перенаправление на окно авторизации»:
 - означает, что cookie-файлы с данными отсутствуют;
 - действия: войти в систему с помощью пароля и логина;
- 3) «Заявка создана»:
 - означает, что заявка успешно создана;
 - действия: не нужны;
- 4) «Не удалось сохранить заявку»:
 - означает, что заявка не была сохранена;
 - действия: проверить корректность введенных данных;
- 5) «Заявка удалена»:
 - означает, что заявка удалена успешно;
 - действия: не нужны.

2 Дипломный проект

2.1 Введение по дипломному проекту

Современные предприятия стремятся повышать эффективность работы за счёт цифровизации и автоматизации внутренних процессов. Особенно это актуально для организаций, связанных с продажей товаров и оказанием услуг, например, компьютерных магазинов, где сотрудники ежедневно выполняют множество операций: учёт товаров, оформление продаж, регистрация ремонтов и взаимодействие с клиентами.

Использование разьединённых документов и таблиц приводит к ошибкам при ручном вводе данных, затрудняет поиск информации, вызывает дублирование записей и снижает скорость обслуживания клиентов. Это негативно влияет на качество работы предприятия.

Наиболее эффективным решением становится создание единой веб-информационной системы. Веб-приложение обеспечивает доступ к системе с любого устройства внутри локальной сети, не требует установки на каждый компьютер, поддерживает многопользовательский режим, упрощает обновление функционала и позволяет чётко разграничивать роли сотрудников, повышая безопасность данных.

Актуальность темы заключается в необходимости разработки удобного и надёжного веб-приложения для автоматизации ключевых бизнес-процессов компьютерного магазина: учёта товаров, управления продажами, регистрации оказанных услуг, ведения базы сотрудников и контроля их действий.

Цель работы – разработать веб-приложение для оптимизации процессов управления предприятием.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать предметную область, выявить основные проблемы текущего процесса работы и определить требования к системе;
- спроектировать архитектуру веб-приложения и выбрать подходящий стек технологий;
- разработать структуру базы данных, обеспечивающую хранение информации о товарах, продажах, услугах, сотрудниках и ролях доступа;
- разработать пользовательский интерфейс, обеспечивающий удобную работу сотрудников;
- реализовать функционал поиска, фильтрации, добавления, редактирования и удаления данных.

2.2 Анализ предметной области

В качестве объекта исследования был взят магазин компьютерной техники «Девайс». Данный магазин функционирует в форме индивидуального предпринимателя. Предприятие занимается продажей компьютерного оборудования, а также оказанием услуг по ремонту и обслуживанию компьютерной техники.

Основными клиентами организации являются частные лица и небольшие организации, которым необходима компьютерная техника.

Структура магазина содержит следующие отделы:

1) отдел продаж – в данном отделе закреплены продавцы, они занимаются консультацией клиентов, оформлением продаж товаров в магазине;

2) сервисный отдел – ремонтники оказывают услуги в виде диагностики, ремонта, настройки и сборки компьютерной техники;

3) склад – управляющий и продавцы обеспечивают хранение товаров, контролирует количество продукции, а также осуществляет учёт поступлений и списаний продукции;

4) административный отдел – включает в себя владельца и управляющего, он контролирует работу сотрудников, ведёт финансовый учёт, а также анализирует продажи и оказанные услуги.

На данном предприятии сотрудники часто выполняют функции разных отделов одновременно. В качестве примера можно привести то, что продавцы могут участвовать как в продаже товаров, так и в их приемке на склад.

Учёт информации в данной организации происходит вручную с использованием бумажных носителей и электронных таблиц. Продавцы и ремонтники фиксируют продажи и услуги отдельно, а данные о количестве товара обновляются периодически. Ведение информации вручную приводит к значительной потере времени на запись данных, высокой вероятности ошибок при вводе, затруднённому доступу к актуальной информации о наличии товаров и замедляет формирование отчетов о деятельности магазина.

В связи с указанными проблемами возникает задача в разработке веб-приложения для оптимизации процессов управления предприятием. Программа позволит обеспечить централизованное хранение и управление данными о товарах, продажах и услугах, предоставит удобный интерфейс для поиска, фильтрации и сортировки, автоматически формировать актуальные отчёты, а также реализует разграничение прав доступа пользователей, что обеспечит безопасность информации и распределение обязанностей между владельцем, продавцами и сотрудниками сервисного отдела.

Внедрение веб-приложения позволит оптимизировать управление предприятием за счёт централизованного хранения данных, а также автоматизации учёта товаров, продаж и услуг. Это ускорит обработку заказов, снизит количество ошибок при работе с информацией и повысит прозрачность деятельности сотрудников. Таким образом, веб-приложение решит актуальные проблемы

магазина «Девайс», обеспечит эффективное управление и повысит производительность работы предприятия.

2.3 Требования к продукту

2.3.1 Функциональные требования

Разработка веб-приложения требует чёткого определения требований к программному продукту. Предприятию необходимо приложение, которое легко развёртывать и поддерживать на всех рабочих местах: установка не должна требовать действий на каждом отдельном компьютере, а обновления системы должны проводиться централизованно на сервере. Кроме того, приложение должно быть кроссплатформенным и корректно работать на разных операционных системах компьютеров. Для реализации этих требований была выбрана платформа веб-приложения. Требования к программному продукту разделяются на функциональные, нефункциональные и эксплуатационные.

Функциональные требования описывают действия, которые система должна выполнять для обеспечения управления магазином и поддержания всех бизнес-процессов. В качестве функциональных требований были выделены следующие:

- учёт товаров. Приложение должно обеспечивать возможность добавления новых товаров, изменения информации о существующих продуктах, а также удаления ненужных товаров. Для каждого товара должны храниться следующие данные: название, категория, описание, цена, количество на складе, поставщик и дата поступления. Эта функциональность позволит обеспечить контроль за наличием товаров и предотвращать дефицит или переполнение склада;

- учёт продаж. Система должна регистрировать все продажи товаров, фиксировать дату, время и продавца. Это позволит отслеживать динамику продаж, выявлять наиболее востребованные позиции и формировать корректные отчёты по выручке;

- учёт услуг. Приложение должно обеспечивать регистрацию всех оказываемых сервисных услуг: ремонт, настройка, диагностика и сборка техники. Для каждой услуги фиксируются мастер, вид услуги, стоимость и дата её оказания. Такая функциональность позволит оценивать загрузку сервисного отдела и контролировать качество работы сотрудников;

- формирование отчётов. Система должна автоматически генерировать отчёты по ключевым направлениям: продажи товаров, остатки на складе, оказанные услуги, выручка и прибыль за выбранный период. Отчёты должны быть доступны для печати и экспорта в популярные форматы (PDF, Excel);

- поиск и фильтрация информации. Пользователи должны иметь возможность выполнять поиск по любым параметрам: название товара, категория,

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		22

сотрудник, дата продажи и т.д. Фильтрация и сортировка данных должны быть реализованы в реальном времени, чтобы ускорить доступ к нужной информации;

- разграничение прав доступа. Система должна поддерживать несколько ролей пользователей: владелец, продавец и сотрудник сервисного отдела.;
- управление складом. Приложение должно отслеживать поступления и списания товаров, контролировать остатки, формировать уведомления о необходимости пополнения запасов. Такой функционал позволит автоматизировать процессы складского учёта и снизить вероятность ошибок.

2.3.2 Нефункциональные требования

Нефункциональные требования описывают свойства и характеристики системы, не связанные напрямую с выполняемыми функциями, но влияющие на качество и удобство использования. В качестве нефункциональных требований были выделены следующие:

- производительность. Приложение должно обеспечивать быстрый отклик на действия пользователя, формировать отчёты и выполнять поиск данных за минимальное время;
- надёжность. Данные должны сохраняться без потерь, а приложение корректно работать при перегрузках. Система должна обеспечивать резервное копирование информации и восстановление данных в случае ошибок;
- интерфейс пользователя. Приложение должно иметь интуитивно понятный интерфейс, доступный для пользователей с разным уровнем подготовки. Основные элементы управления должны быть визуально выделены, а навигация по системе – простой и логичной;
- совместимость. Веб-приложение должно корректно работать в современных браузерах (Yandex Browser, Google Chrome, Microsoft Edge).

2.3.3 Эксплуатационные требования

Эксплуатационные требования определяют условия использования приложения, удобство его обслуживания и поддержку пользователей. В качестве эксплуатационные требования были выделены следующие:

- поддержка пользователей. Для всех категорий пользователей должны быть предоставлены инструкции и справочная информация по работе с приложением.
- обновляемость. Возможность установки обновлений и новых версий системы без прерывания работы магазина;

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		23

- безопасность данных. Система должна обеспечивать защиту конфиденциальной информации;
- администрирование. Приложение должно предоставлять инструменты для настройки прав доступа, добавления новых пользователей и контроля активности сотрудников.

Минимальные требования для клиента:

- операционная система: Windows 10;
- процессор: Intel Pentium 4 или выше;
- оперативная память: минимум 2 ГБ;
- жесткий диск с свободным местом на диске: 1 ГБ;
- браузер: Google Chrome, Яндекс Браузер, Microsoft Edge.

Рекомендованные требования для клиента:

- операционная система: Windows 10, Windows 11;
- процессор: двухъядерный Intel Core i3 или аналог;
- оперативная память: 4 ГБ и более;
- видеокарта: поддержка DirectX 11;
- SSD-накопитель с свободным местом на диске: 1 ГБ;
- браузер: Google Chrome, Яндекс Браузер, Microsoft Edge.

Минимальные требования для сервера:

- операционная система: Windows 10;
- процессор: Intel Core i3 или аналогичный (2 ядра, 2.0 ГГц);
- СУБД: PostgreSQL 13;
- среда выполнения: ASP.NET Core 6.0;
- оперативная память: минимум 4 ГБ;
- жесткий диск с свободным местом на диске: 20 ГБ;
- браузер: Google Chrome, Яндекс Браузер, Microsoft Edge.

Рекомендованные требования для сервера:

- операционная система: Windows 10, Windows 11;
- процессор: Intel Core i5 / AMD Ryzen 5 (4 ядра, 3.0 ГГц и выше);
- СУБД: PostgreSQL 15;
- среда выполнения: ASP.NET Core 6.0;
- оперативная память: 8-16 ГБ;
- жесткий диск с свободным местом на диске: 50 ГБ;
- браузер: Google Chrome, Яндекс Браузер, Microsoft Edge.

2.4 Обоснование выбора инструментальных средств разработки

2.4.1 Язык программирования

Разработка программного продукта требует осознанного выбора инструментальных средств, так как от этого напрямую зависят функциональные

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		24

возможности и надёжность системы. В связи с этим был проведён анализ возможных технологий и инструментов разработки. При выборе учитывались такие критерии, как стабильность работы и поддержка многопользовательского режима, бесплатность, совместимость компонентов между собой, а также удобство разработки и тестирования приложения.

В соответствии с требованиями предприятия магазина «Девайс», система должна обеспечивать централизованный учёт товаров, продаж и услуг с доступом для всех сотрудников. Для реализации этих задач был выбран подход веб-приложения. Веб-архитектура позволяет поддерживать многопользовательский режим и синхронизацию данных в реальном времени, упрощает обновление и сопровождение системы, обеспечивает кроссплатформенность и перспективы масштабирования, что делает её более подходящей для условий работы предприятия по сравнению с традиционным десктопным приложением.

На этапе проектирования рассматривались два основных варианта языка разработки – C# (с использованием ASP.NET Core) и Python (с использованием Django).

C# – это современный объектно-ориентированный язык программирования, разработанный компанией Microsoft. Он отличается высокой производительностью, безопасностью и строгой типизацией. C# широко используется для разработки настольных, мобильных и веб-приложений.

ASP.NET Core – это кроссплатформенный фреймворк от Microsoft для создания веб-приложений, API и сервисов. Он обеспечивает высокую производительность, масштабируемость и безопасность, а также поддерживает архитектуру Model-View-Controller (MVC), что делает код более структурированным и удобным для сопровождения.

Достоинства C# и ASP.NET Core:

- высокая производительность и оптимизация под серверные приложения;
- встроенные механизмы безопасности и аутентификации;
- поддержка шаблона архитектуры MVC (Model-View-Controller);
- хорошая интеграция с различными СУБД;
- возможность развертывания в локальной сети без необходимости подключения к Интернету.

Недостатки C# и ASP.NET Core:

- сравнительно более высокая сложность начальной настройки проекта;
- требовательность к ресурсам при работе на слабых серверах.

Python – интерпретируемый язык программирования общего назначения, отличающийся простым синтаксисом и широким набором библиотек. Он популярен для задач анализа данных, машинного обучения и создания веб-приложений.

Django – это фреймворк на языке Python, предназначенный для быстрой разработки веб-приложений. Django включает встроенную ORM, систему аутентификации, панель администратора и множество инструментов для безопасной и быстрой разработки.

Достоинства Python и Django:

- простота и быстрота разработки;
- большое количество сторонних библиотек;
- кроссплатформенность и гибкость.

Недостатки Python и Django:

- более низкая производительность при большом числе запросов;
- сложность организации безопасной многопользовательской среды в локальной сети;
- необходимость ручной оптимизации для работы с крупными базами данных.

После сравнения языков C# и Python предпочтение было отдано C# с использованием ASP.NET Core, так как данная платформа лучше подходит для корпоративных решений, обеспечивает высокую производительность, встроенную безопасность и надёжную работу в локальной сети.

Для взаимодействия с базой данных используется Entity Framework Core (EF Core) – современный ORM-фреймворк, который позволяет работать с данными через объектные модели без прямого написания SQL-запросов. Он поддерживает автоматическую генерацию миграций, упрощает обновление структуры базы данных и сокращает вероятность ошибок при работе с записями.

2.4.2 Система управления базами данных

Для хранения данных рассматривались три наиболее распространённые СУБД: MySQL, Microsoft SQL Server и PostgreSQL.

MySQL – одна из самых популярных систем управления базами данных, также распространяемая бесплатно (в базовой версии). Она отличается простотой администрирования и хорошей скоростью при работе с небольшими объёмами данных.

Достоинства MySQL:

- высокая скорость обработки запросов;
- простота установки и использования;
- наличие большого сообщества разработчиков.

Недостатки MySQL:

- ограниченные возможности для работы со сложными структурами данных;
- коммерческая лицензия для корпоративного использования;
- менее гибкая система разграничения прав пользователей.

Microsoft SQL Server – коммерческая СУБД от Microsoft, предназначенная для корпоративных решений. Она обладает высокой производительностью и надёжной системой безопасности, но требует покупки лицензии.

Достоинства Microsoft SQL Server:

- высокая производительность и интеграция с Visual Studio;

- поддержка сложных транзакций и хранимых процедур;
- надёжность и удобство резервного копирования.

Недостатки Microsoft SQL Server:

- необходимость приобретения лицензии;
- высокая нагрузка на систему при ограниченных ресурсах.

PostgreSQL – это мощная объектно-реляционная система управления базами данных с открытым исходным кодом. Она поддерживает сложные запросы, триггеры, процедуры и транзакции, что делает её подходящей как для малых, так и для крупных проектов. PostgreSQL отличается стабильностью, высокой производительностью и отсутствием лицензионных ограничений.

Преимущества PostgreSQL:

- полностью бесплатная и кроссплатформенная;
- поддержка сложных запросов, триггеров и транзакций;
- высокая надёжность и отказоустойчивость;
- наличие удобного интерфейса pgAdmin для администрирования;
- простота взаимодействия с ASP.NET Core через драйвер Npgsql.

Недостатки PostgreSQL:

- относительно сложная начальная настройка;
- чуть более высокая нагрузка на память по сравнению с MySQL.

С учётом требований проекта и необходимости работы в локальной сети предприятия, была выбрана PostgreSQL. Она обеспечивает надёжное хранение данных, стабильную работу при больших объёмах информации и полную совместимость с выбранным стеком технологий (C# + ASP.NET Core + EF Core).

2.4.3 Среда разработки

Разработка приложения осуществляется в интегрированной среде Microsoft Visual Studio 2022, которая предоставляет разработчику все необходимые инструменты для написания, тестирования и отладки программного кода.

Visual Studio обладает следующими преимуществами:

- мощная система отладки и тестирования;
- поддержка системы контроля версий Git;
- встроенный менеджер пакетов NuGet для подключения библиотек;
- готовые шаблоны проектов ASP.NET Core;
- удобный интерфейс и высокая производительность работы.

Для администрирования базы данных применяется pgAdmin 4. pgAdmin 4 – это графический инструмент для работы с базой данных PostgreSQL. Этот инструмент позволяет:

- просматривать структуру БД и связи между таблицами;
- выполнять SQL-запросы и анализировать результаты;
- управлять пользователями и правами доступа;
- контролировать производительность и состояние сервера PostgreSQL.

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		27

2.4.4 Проектирование интерфейса

Для предварительной визуализации интерфейсов приложения используется онлайн-сервис Figma. Он позволяет создавать макеты экранов и визуальные элементы будущей программы.

Достоинства Figma:

- простое создание интерактивных прототипов интерфейса;
- визуальное согласование дизайна с реальной структурой приложения;
- наличие бесплатного тарифа для пользования;
- ускорение этапа разработки интерфейса в Visual Studio.

Таким образом, использование Figma позволяет заранее продумать внешний вид и удобство работы с системой.

2.4.5 Клиентская часть приложения

Интерфейс веб-приложения реализуется с помощью стандартных технологий:

- HTML5 (для структурирования веб-страниц);
- CSS3 (для оформления и стилизации элементов);
- JavaScript (для интерактивного взаимодействия пользователя с системой);
- Bootstrap (для адаптивной вёрстки и унифицированного дизайна).

Такое сочетание технологий обеспечивает современный внешний вид, простоту восприятия и корректное отображение интерфейса.

2.4.6 Вывод по выбору инструментов

В ходе проведённого анализа различных инструментальных средств, применяемых при разработке программных продуктов, было принято решение использовать язык C# вместе с платформой ASP.NET Core, систему управления базами данных PostgreSQL, интегрированную среду разработки Microsoft Visual Studio, а также инструменты pgAdmin и Figma для администрирования и проектирования. Такой выбор обеспечивает высокую производительность, стабильность и безопасность работы приложения, удобство разработки и сопровождения, а также возможность масштабирования и интеграции с другими системами.

2.5 Разработка структуры веб-приложения

Разработка программы начинается с проектирования структуры веб-приложения, отражающей взаимодействие между основными компонентами системы. которая состоит из трёх основных компонентов:

- 1) клиентская часть (frontend);
- 2) серверная часть (backend);
- 3) база данных (database).

Клиентская часть представляет собой веб-интерфейс, доступный через браузер, что позволяет пользователям работать с системой без установки дополнительного программного обеспечения. Интерфейс создаётся с использованием HTML, CSS и JavaScript.

Пользовательская часть отвечает за ввод данных, отображение информации и передачу запросов на сервер. Все действия пользователя (добавление записей, редактирование, поиск и фильтрация) преобразуются в запросы к серверу с помощью HTTP-протокола.

Серверная часть реализована на платформе ASP.NET Core, которая отвечает за приём запросов от клиента, обработку данных и передачу результатов обратно пользователю. В ней реализуется основная бизнес-логика приложения – управление товарами, продажами, услугами и пользователями.

Сервер также обеспечивает аутентификацию и авторизацию пользователей, разграничение прав доступа и валидацию данных. Взаимодействие с базой данных осуществляется с использованием технологии Entity Framework Core, которая позволяет работать с данными через объектные модели и автоматически формировать SQL-запросы.

Хранение данных осуществляется в системе PostgreSQL, которая обеспечивает надёжность, отказоустойчивость и безопасность. Структура базы данных включает взаимосвязанные таблицы: пользователи, товары, продажи, услуги, выполненные услуги, поставки и поставщики. Для поддержания целостности данных используются первичные и внешние ключи, а также проверки ограничений.

Обмен данными между уровнями приложения организован по принципу «запрос-обработка-ответ». Пользователь выполняет действие на клиенте (например, поиск товара). Клиентская часть формирует HTTP-запрос и передаёт его на сервер. Сервер обрабатывает запрос, обращается к базе данных при необходимости и получает результат. Ответ возвращается в формате JSON, после чего отображается на экране пользователя.

Взаимодействие компонентов веб-приложения представлено в блок-схеме, которая отображена на рисунке 2.1.

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		29



Рисунок 2.1 – Блок-схема структуры приложения

2.6 Проектирование и разработка базы данных

Проектирование базы данных является одним из важнейших этапов разработки приложения, поскольку вся логика и интерфейс приложения основываются на базе данных. На данном этапе была сформирована схема база данных, а также была разработана сама база данных.

Структура базы данных, состоящая из сущностей и их атрибутов:

1) Users (пользователи):

- user_id (идентификатор пользователя);
- login (уникальный логин);
- password_hash (пароль);
- full_name (полное имя);
- phone (телефон);
- role_id (ссылка на роль пользователя);
- is_active (признак активности записи, позволяющий не удалять сотрудника физически и сохранить историю продаж).

2) Roles (роли):

- role_id (идентификатор роли);
- role_name (имя роли).

3) Products (товары):

- product_id (идентификатор товара);
- name (наименование);
- description (описание);
- price (цена);
- stock_quantity (количество на складе);

– is_active (позволяет скрывать устаревшие позиции).

4) Sales (продажи):

- sale_id (идентификатор продажи);
- seller_id (идентификатор продавца);
- sale_date (дата продажи);
- total_amount (общая сумма продажи).

5) SaleItems (товарные позиции продажи):

- id (идентификатор записи);
- sale_id (идентификатор продажи);
- product_id (идентификатор товара);
- quantity (количество товара);
- price_at_sale (цена товара на момент продажи).

6) Services (услуги):

- service_id (идентификатор услуги);
- name (название услуги);
- description (описание);
- price (стоимость услуги);
- is_active (признак активности услуги).

7) ServiceOrders (заказы услуг):

- order_id (идентификатор заказа);
- master_id (идентификатор мастера);
- order_date (дата выполнения услуги);
- total_amount (итоговая стоимость заказа);
- notes (комментарий).

8) ServiceOrderItems (услуги в заказе):

- id (идентификатор записи);
- order_id (идентификатор заказа);
- service_id (идентификатор услуги);
- price_at_order (цена услуги).

Связи между сущностями базы данных отражают реальные процессы работы магазина и сервисного центра. Один пользователь может выполнить множество продаж, поэтому связь «Пользователь – Продажи» реализована как отношение 1 ко многим. Каждая продажа включает несколько товарных позиций, что обеспечивает связь «Продажа – Товарные позиции продажи», а каждый товар может встречаться в разных продажах, формируя связь «Товар – Товарные позиции продажи». Аналогично организован учёт услуг: один заказ может содержать несколько услуг («Заказ услуг – Услуги заказа»), а каждая услуга может быть выполнена во множестве заказов («Услуга – Услуги заказа»). Кроме того, один мастер может выполнить множество сервисных заказов, что отражено в связи «Пользователь – Заказы услуг». Такая структура позволяет корректно фиксировать продажи, выполнение услуг и движение товаров, сохраняя детализацию каждой операции.

ERD-диаграмма, отражающая структуру базы данных представлена на рисунке 2.2.

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		31

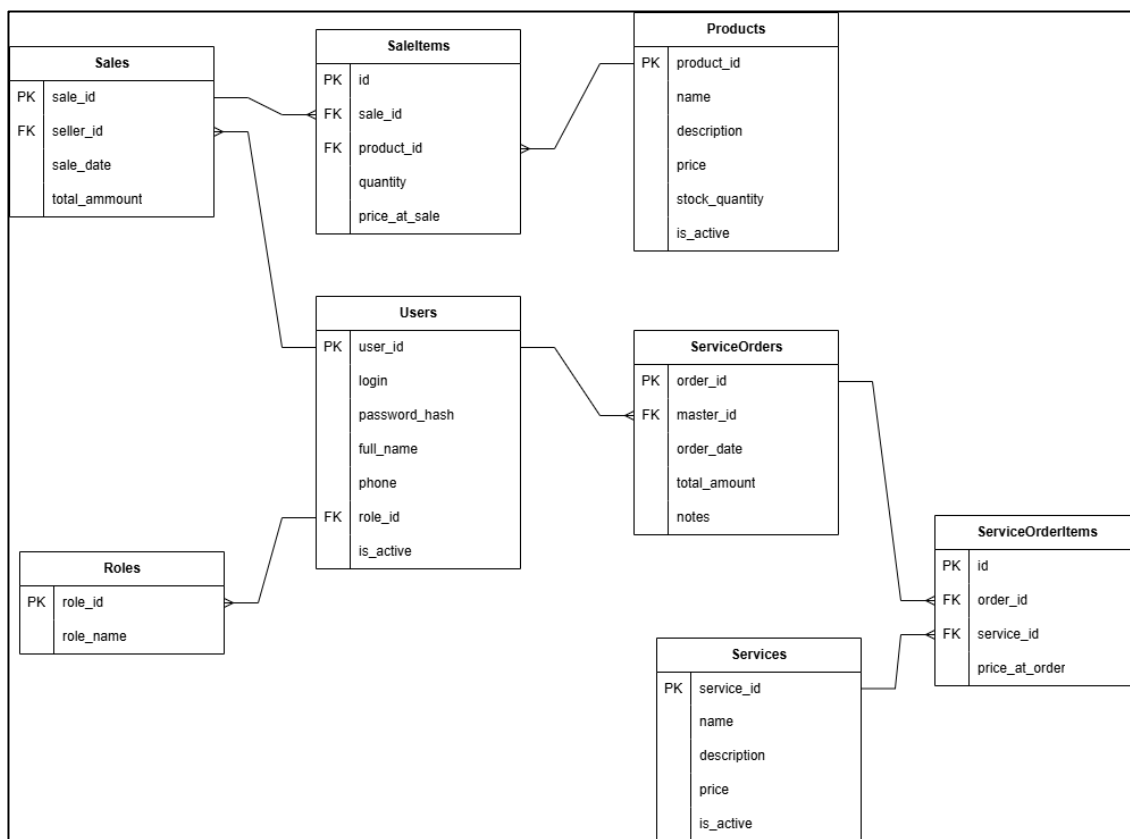


Рисунок 2.2 – ERD-диаграмма

2.7 Разработка пользовательского интерфейса

Разработка пользовательского интерфейса является важным этапом создания веб-приложения, так как от удобства и понятности интерфейса зависит эффективность работы сотрудников магазина. На предыдущем этапе были сформированы макеты основных страниц в сервисе Figma, которые определили структуру экранов, расположение элементов и общую логику взаимодействия пользователя с системой. На этапе реализации эти макеты были преобразованы в полноценные веб-страницы с использованием HTML, CSS, JavaScript и фреймворка Bootstrap.

Для оформления дизайна использовался фреймворк Bootstrap и технология Razor Pages, обеспечивающий адаптивность и единый стиль всех элементов. Благодаря встроенным компонентам Bootstrap (панели навигации, таблицы, кнопки, модальные окна), реализация интерфейса значительно ускорилась, а внешний вид форм стал структурированным и современным. Цветовая схема и расположение основных элементов были сохранены в соответствии с макетами, а взаимодействие пользователя с системой стало интуитивно понятным.

В процессе реализации были разработаны следующие основные элементы пользовательского интерфейса:

- форма авторизации, включающая поля ввода логина и пароля, а также обработку ошибок при некорректном вводе данных;
- главная страница, содержащая навигационное меню и таблицы данных, доступные пользователю в соответствии с его ролью;
- страницы работы с товарами, включая просмотр списка товаров, поиск, фильтрацию и формы добавления или редактирования записей;
- страницы регистрации продаж и услуг, разработанные на основе выпадающих списков и элементов выбора, что обеспечивает простоту работы и минимизацию ошибок;
- таблицы отчётности, оформленные в едином стиле и обеспечивающие наглядное представление данных руководителю предприятия.

Полученные страницы веб-приложения показаны на рисунках 2.3, 2.4.

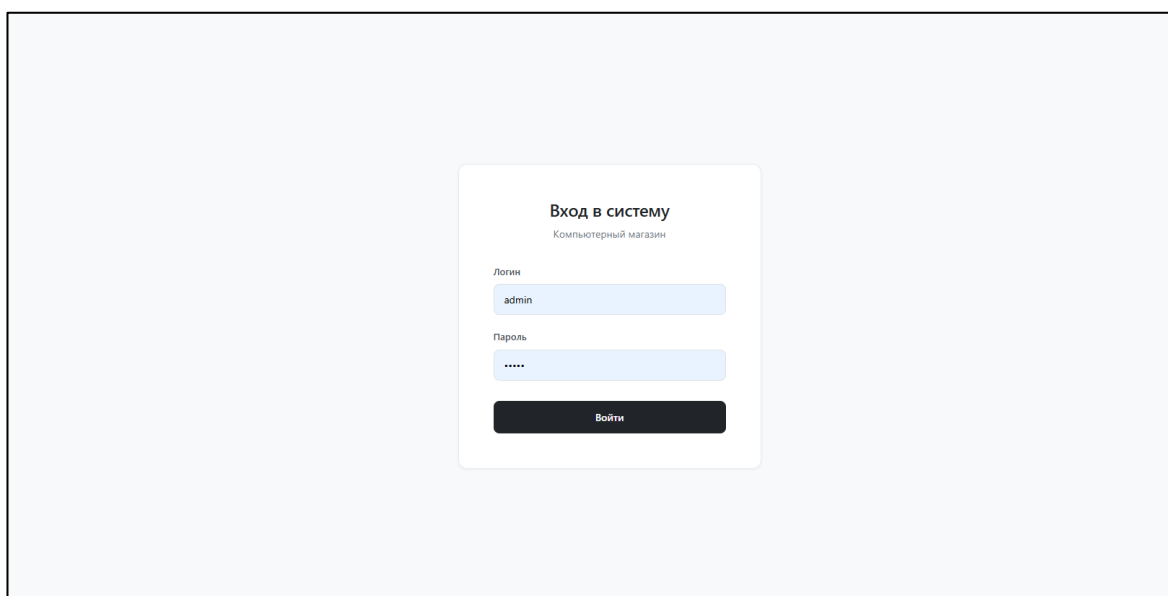


Рисунок 2.3 – Полученное окно авторизации

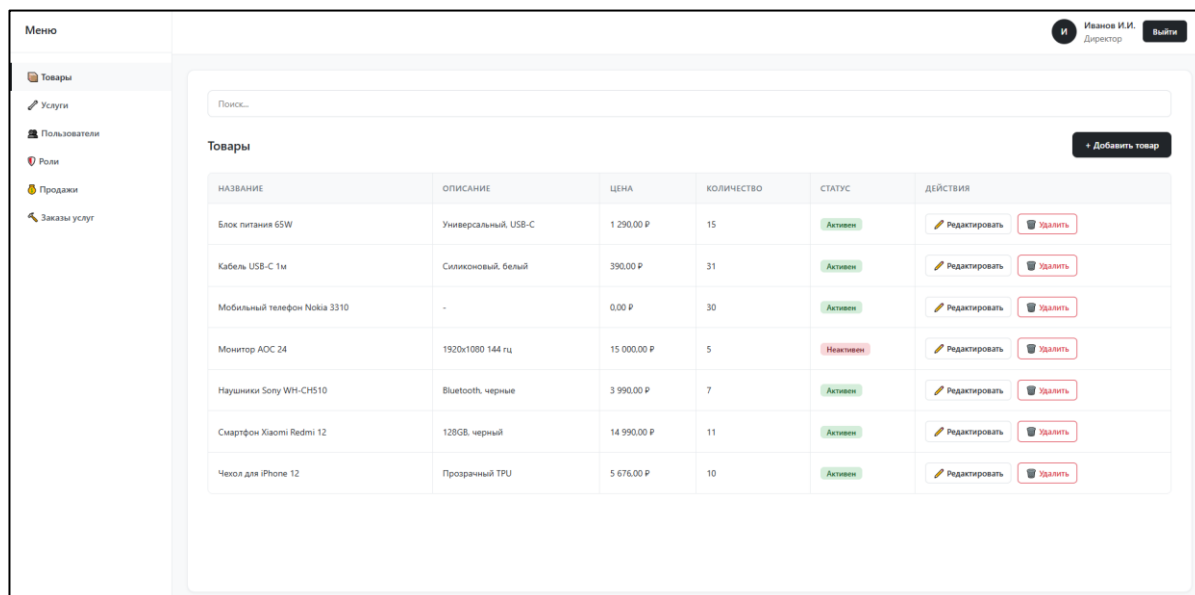


Рисунок 2.4 – Полученная основная страница

Инициализация и запуск приложения были выполнены в файле Program.cs (Приложение Б). Сначала был создан базовый объект для настройки проекта. После этого были подключены основные службы, такие как страницы Razor, доступ к базе данных PostgreSQL, а также система входа в систему через cookie.

После подключения служб был настроен порядок обработки запросов. На этом этапе включается обработка ошибок, загрузка статических файлов, а также система маршрутов, отвечающая за переходы между страницами. В конце страницы Razor были подключены к маршрутам, чтобы они стали доступны пользователю.

Контекст базы данных был реализован в файле ApplicationDbContext.cs (Приложение Б). В этом классе была задана связь между приложением и таблицами базы данных. Через него приложение получает доступ к таким сущностям, как пользователи, роли, товары, услуги, продажи и связанные с ними элементы.

В классе были объявлены наборы данных (DbSet), каждый из которых соответствует отдельной таблице в базе. Это позволило выполнять операции чтения, добавления, изменения и удаления записей. В методе OnModelCreating() была выполнена дополнительная настройка модели, чтобы правильно связать классы приложения с таблицами в базе данных.

Для удобства и лучшего разделения структуры проекта все модели данных были размещены в отдельных файлах. Каждая модель соответствует своей таблице в базе данных и использует Entity Framework для работы с записями. Одной из таких моделей стала Service, код которой представлен в приложении А.

В этой модели описываются основные поля услуги: уникальный идентификатор, название, необязательное описание, стоимость и статус активности. Класс служит шаблоном, по которому Entity Framework создаёт таблицу и управляет её содержимым во время работы приложения.

Страница авторизации была реализована из двух частей: визуальной (Login.cshtml) и логики обработки (Login.cshtml.cs). Логика обработки отвечает за проверку данных пользователя и вход в систему. На странице обрабатываются введенные логин и пароль, выполняются проверка их корректности и поиск соответствующего пользователя в базе данных.

Если пользователь был найден и пароль совпал, то система формирует данные для входа и создает cookie для аутентификации, благодаря которому сотрудник остается авторизованным в течение заданного времени. В случае ошибок – например, если логин или пароль были неверными – на странице выводится соответствующее уведомление. После успешной авторизации пользователь перенаправляется на главную страницу приложения. Код данной части программы представлен в приложении А.

Для всех таблиц базы данных в приложении был реализован общий функционал работы с данными. Приложение загружает записи из базы с возможностью поиска и фильтрации, позволяет создавать новые записи и редактировать существующие, а также проверяет корректность введенных данных. Например, для товаров можно добавлять новые позиции, изменять название, описание, цену, количество на складе и статус активности, а для продаж – проверять наличие товаров и фиксировать покупки с сохранением всех позиций в базе.

Все операции выполняются с учетом целостности данных: изменения вносятся через транзакции, чтобы при возникновении ошибок все действия откатывались. После выполнения операций пользователи автоматически возвращаются на соответствующую вкладку приложения. Код некоторых методов и для работы с таблицами приведен в приложении Б.

В результате пользовательский интерфейс получился удобным, понятным и визуально целостным, а его структура соответствует изначальным макетам. Это обеспечивает комфортную работу сотрудников с системой и позволяет быстро выполнять основные операции магазина.

2.8 Доработка приложения

2.8.1 Дополнительные требования к продукту

Доработка программного продукта направлена на повышение эффективности управления товарными запасами, поставками и аналитикой информации, а также на обеспечение разграничения прав доступа пользователей.

В рамках доработки программного продукта были определены следующие дополнительные функциональные требования:

– уведомления о малом количестве товара (система должна автоматически отслеживать остатки товаров на складе и отображать уведомления при достижении малого уровня запасов);

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		35

- оформление и хранение поставок (приложение должно обеспечивать возможность создания, хранения и просмотра информации о поставках товаров, а также автоматическое обновление складских остатков при поступлении товаров);
- расширенное формирование отчётов (система должна формировать отчёты по продажам, товарам и оказанным услугам);
- разграничение прав доступа пользователей (веб-приложение должно ограничивать доступ к разделам в зависимости от роли пользователя)

В процессе доработки программного продукта были уточнены следующие нефункциональные требования:

- система должна обеспечивать корректную работу проверок прав доступа без снижения производительности;
- время формирования отчётов и уведомлений должно быть минимальным;
- интерфейс пользователя должен адаптироваться под различные роли, отображая только доступные разделы и функции.

Эксплуатационные требования не изменились.

2.8.2 Блок-схемы новых функций

Перед разработкой функции уведомления о малом количестве товара была разработана блок-схема, представленная на рисунке 2.5.

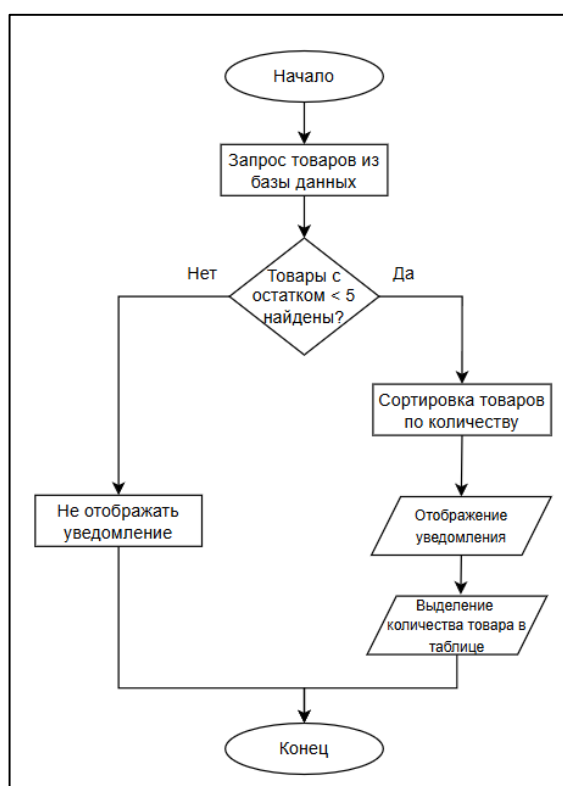


Рисунок 2.5 – Блок-схема уведомлений

Данная блок-схема описывает принцип работы уведомлений. Он начинается с того, что происходит запрос товаров из базы данных и проверяется количество единиц товара. В случае, если товара равно или больше 5 штук, то уведомление не будет отображаться. Если же товара меньше 5 единиц, то данный берётся данный товар и его количество сортируется, чтобы в ситуации, когда товаров несколько первым отображалась позиция товара с наименьшим количеством. После этого в нижней части меню вкладок отображается соответствующее уведомление и выделяется количество товара в таблице.

Следующей функцией стало формирование отчётов. Её блок-схема изображена на рисунке 2.6.

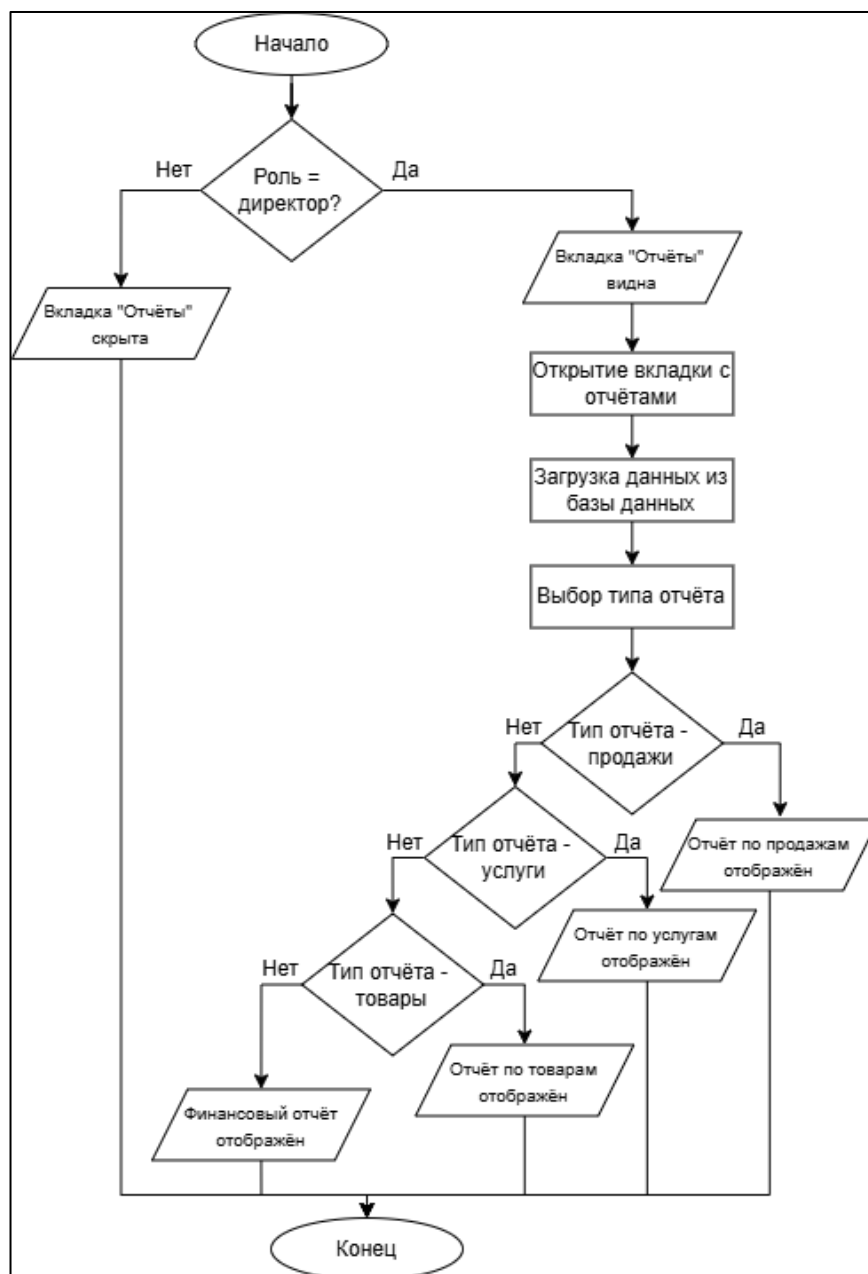


Рисунок 2.6 – Блок-схема отчётов

Формирование отчётов начинается с проверки роли пользователя. Доступ к вкладке отчётов доступен только директору, другим пользователям она не видна. После проверки роли, если она соответствует, пользователь открывает вкладку с отчётами и происходит загрузка данных из базы данных. Исходя из этой информации формируется самый первый тип отчёта – по продажам. Затем пользователь может выбрать подходящий вид отчёта и согласно ему, будет сформирован и отображён соответствующий отчёт. Параллельно пользователь может выбрать диапазон даты формирования отчёта, по которому создаётся отчёт.

Блок-схема функции формирования и хранения поставок показана на рисунке 2.7.

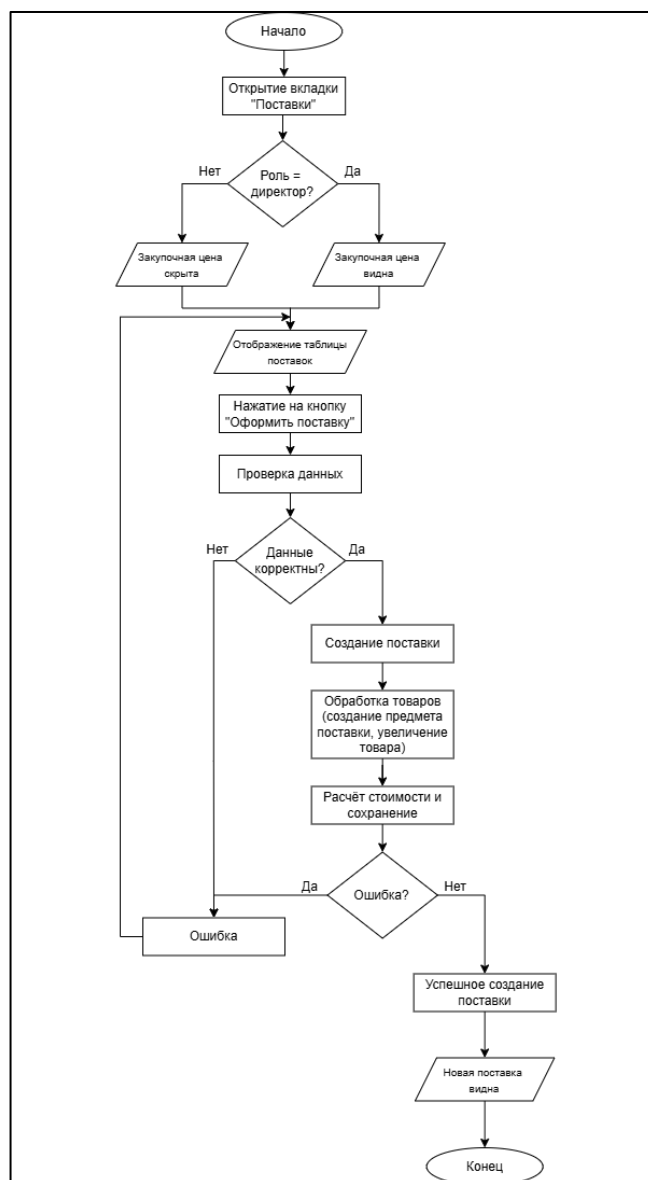


Рисунок 2.7 – Блок-схема поставок

Раздел поставок начинается с открытия вкладки «Поставки». Параллельно происходит проверка роли пользователя. В случае, если пользователем является директор – цены закупки будут отображены, иначе никаких цен пользователь не увидит. После открытия вкладки пользователю отображается таблица поставок. В случае если пользователю необходимо добавить поставку, он нажимает кнопку «Оформить поставку». В этот момент происходит проверка данных о товарах, в случае если всё успешно, то список товаров для выбора отображён. Затем пользователь создаёт поставку, выбирая товары и их количество. После внесения всей продукции поставки и нажатия на кнопку оформления происходит обработка товаров и расчёт стоимости. В случае если все данные отправлены успешно – поставка сохраняется, количество товара на складе меняется, и поставка сохраняется в таблице.

Блок-схема функции разграничения прав доступа показана на рисунке 2.8.



Рисунок 2.8 – Блок-схема разграничения прав доступа

Логика функции разграничения прав доступа заключается в проверке роли пользователя. В случае, если к роли пользователя прописан доступ к вкладке, то у него будет отображаться вкладка. Такая проверка действует на вкладки, в некоторых случаях даже применяется проверка для ограничения функционала, такого как редактирование и удаление записей в таблицах.

2.9 Разработка дополнительного функционала

Внешний вид разработанных уведомлений показан на рисунках 2.9, 2.10.

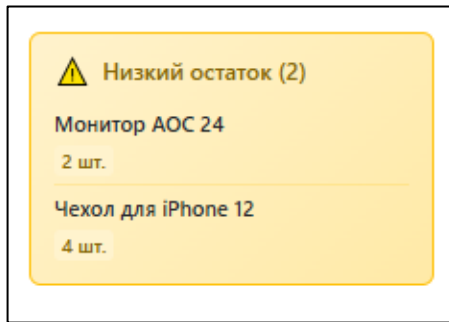


Рисунок 2.9 – Вид уведомления

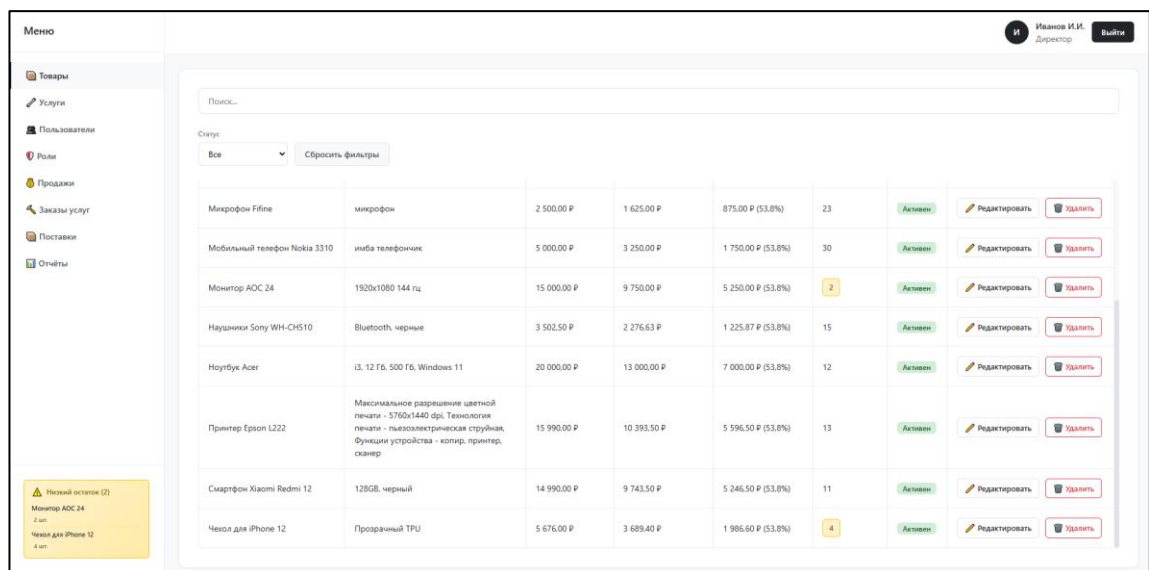


Рисунок 2.10 – Общий вид страницы с уведомлением

Согласно рисункам, можно увидеть, что в качестве места под уведомление была выбрана нижняя часть меню вкладок. Само уведомление отображает название товара и его количество на складе. Для удобства была сделана сортировка товара по количеству, благодаря этому первыми в уведомлениях отмечаются товары с наименьшим количеством. Также параллельно с сообщениями в таблице товаров выделяется количество товара, которого мало.

Внешний вид разработанного раздела с отчётами представлен на рисунках 2.11, 2.12.

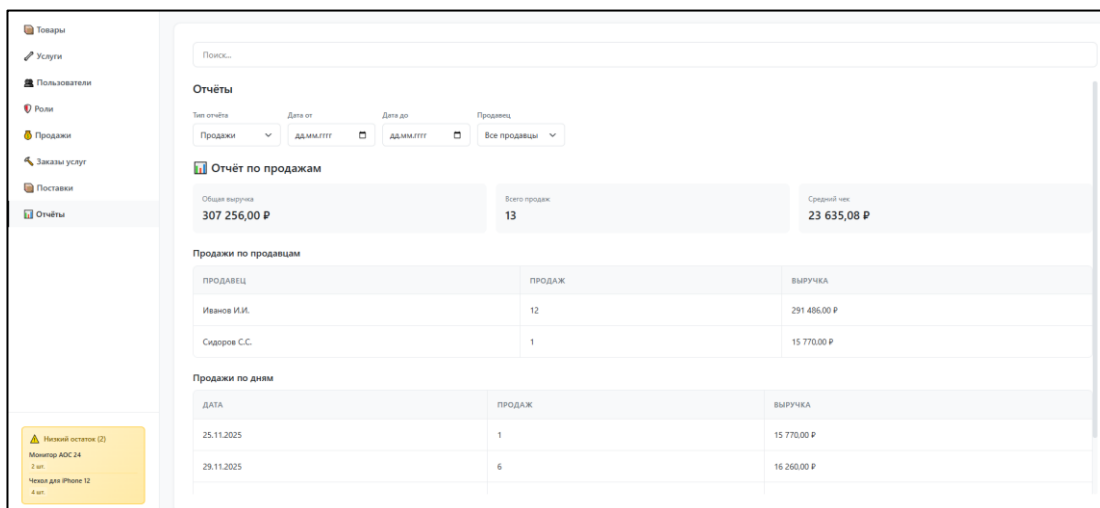


Рисунок 2.11 – Вид раздела с отчётами

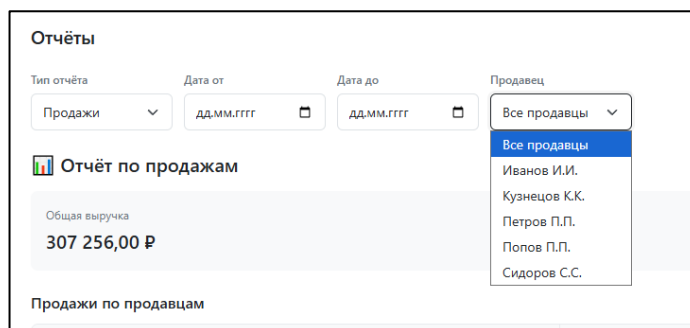


Рисунок 2.12 – Сортировка отчётов

Функция формирования отчётов позволяет директору анализировать деятельность организации. Всего в данном разделе присутствуют 4 типа отчётов: отчёт по продажам, отчёт по услугам, отчёт по товарам и финансовый отчёт. В зависимости от выбранного отчёта отображаются соответствующие данные. Помимо выбора типа отчёта есть возможность выбора периода, за который формируется отчёт. Также была реализована возможность формирования отчёта по деятельности сотрудника, то есть директор может выбрать определенного сотрудника и посмотреть, насколько он был продуктивен во время работы.

Внешний вид разработанного раздела с поставками представлен на рисунках 2.13, 2.14.

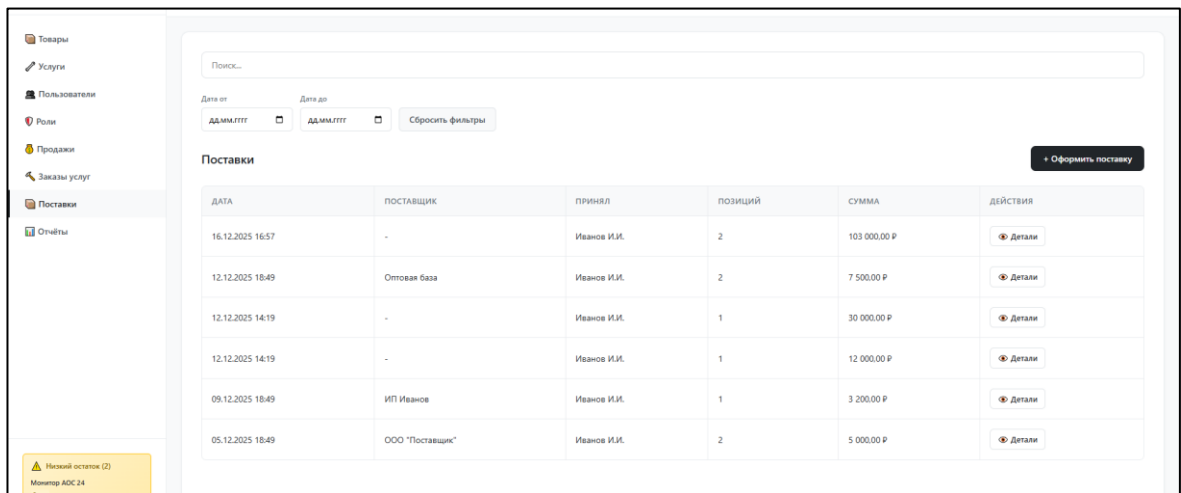


Рисунок 2.13 – Раздел поставок

Рисунок 2.14 – Оформление поставок

Функция формирования и хранения поставок позволяет пополнять запасы товаров и хранить историю их пополнения. В разделе поставок отображена таблица с поставками, возможность сортировки данной таблицы по дате, поиск по поставкам и функция оформления новой поставки. При нажатии на кнопку «Оформить поставку» открывается окно для оформления поставки. В нём пользователь может заполнить поставщика, товары и их количество, а также закупочную цену в случае, если роль пользователя – директор. После успешного оформления поставки продукция добавляется в таблицу с товарами.

Внешний вид приложения пользователей с разными ролями представлен на рисунках 2.15-2.18.

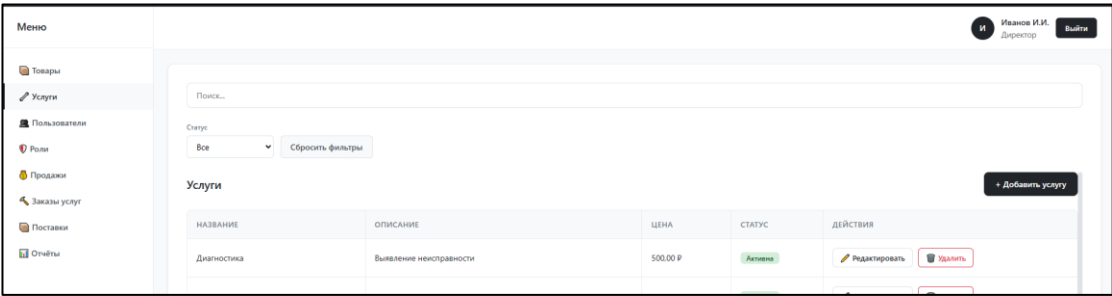


Рисунок 2.15 – Окно приложения у директора

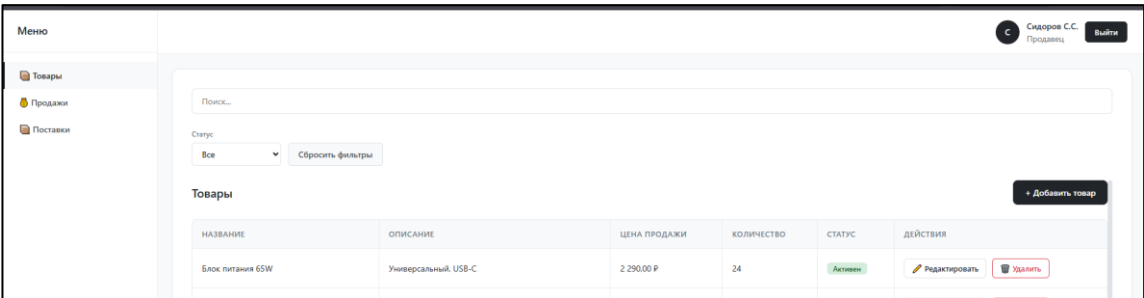


Рисунок 2.16 – Окно приложения у продавца

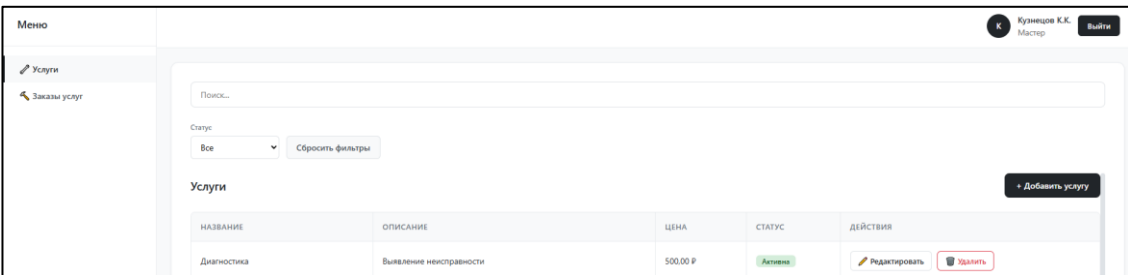


Рисунок 2.17 – Окно приложения у мастера

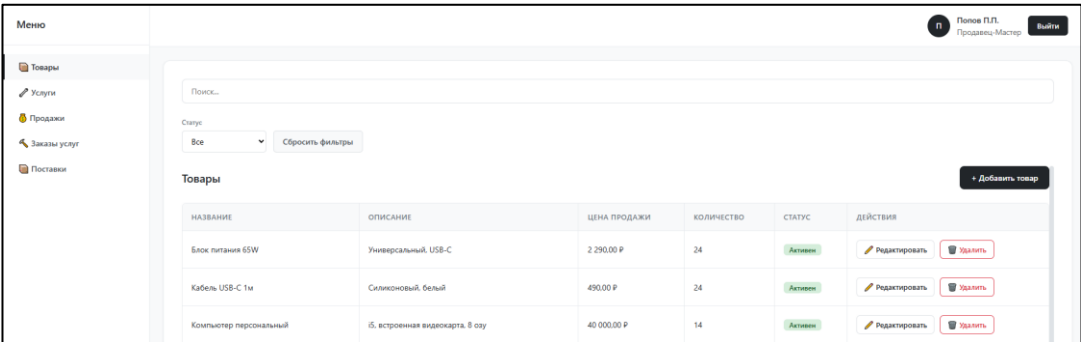


Рисунок 2.18 – Окно приложения у продавца-мастера

Как видно по рисункам, функция разграничения прав доступа обеспечивает безопасность системы путём ограничения доступа пользователей к различным разделам и операциям в зависимости от их роли. Также, в случае если пользователь впишет адрес к вкладке, которая ему недоступна, он не получит доступа к ней и будет перенаправлен на его первую вкладку.

Система поддерживает 5 ролей:

- 1) директор – полный доступ ко всем функциям;
- 2) заместитель – доступ ко всем функциям, кроме удаления пользователей и изменения/удаления ролей;
- 3) продавец – доступ к товарам, продажам и поставкам;
- 4) мастер – доступ к услугам и заказам услуг;
- 5) продавец-мастер – доступ к товарам, услугам, продажам, заказам услуг и поставкам.

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		44

Заключение

В ходе выполнения учебной практики были выполнены все поставленные задачи. Главным итогом работы стала полученная программа и документация к варианту демонстрационного экзамена, а также реализация практического модуля к дипломному проекту.

Для достижения этого результата, были выполнены все этапы разработки:

- проектировка базы данных и написание кода;
- настройка безопасности и роли;
- создание интерфейса удобным и защищенным;
- оформил документацию.

Весь цикл работы сохранен в системе контроля версий Git. Созданные программные продукты работают стабильно и решают задачи организаций.

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		45

Библиография

- 1 С# (Си Шарп, C Sharp): что это за язык программирования, что на нем пишут и его преимущества [Электронный ресурс] – Режим доступа: <https://vc.ru/dev/836215-c-si-sharp-c-sharp-chto-eto-za-yazyk-programmirovaniya-chto-na-nem-pishut-i-ego-preimushstva>
- 2 Документация к PostgreSQL [Электронный ресурс] – Режим доступа: <https://postgrespro.ru/docs/postgresql/current/index>
- 3 Руководство по ASP.NET Core [Электронный ресурс] – Режим доступа: <https://metanit.com/sharp/aspnet6>
- 4 Руководство по Entity Framework Core [Электронный ресурс] – Режим доступа <https://metanit.com/sharp/efcore/>
- 5 HTML: основные понятия, теги и структура для веб-разработки [Электронный ресурс] – Режим доступа: <https://sky.pro/wiki/html/osnovnye-ponyatiya-i-terminy-v-html/>
- 6 CSS для начинающих: основы стилизации HTML от базы до практики [Электронный ресурс] – Режим доступа: <https://sky.pro/wiki/html/osnovy-css-dlya-stilizacii-html/>
- 7 Веб-разработка с нуля: руководство от HTML до JavaScript [Электронный ресурс] – Режим доступа: <https://sky.pro/wiki/html/osnovy-html-css-i-javascript/>
- 8 Руководство по Razor Pages [Электронный ресурс] – Режим доступа: <https://metanit.com/sharp/razorpages/>
- 9 Bootstrap Documentation [Электронный ресурс] – Режим доступа: <https://getbootstrap.com/docs/>
- 10 jQuery API Documentation [Электронный ресурс] – Режим доступа: <https://api.jquery.com/>

Приложение А
(справочная)
Листинг программы

Запросы на создание и заполнение базы данных

```
CREATE TABLE Roles (
    roleID SERIAL PRIMARY KEY,
    roleName VARCHAR(50) NOT NULL
);

CREATE TABLE Statuses (
    statusID SERIAL PRIMARY KEY,
    statusName VARCHAR(50) NOT NULL
);

CREATE TABLE Users (
    userID SERIAL PRIMARY KEY,
    fio VARCHAR(150) NOT NULL,
    phone VARCHAR(20),
    login VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(50) NOT NULL,
    roleID INTEGER NOT NULL,
    FOREIGN KEY (roleID) REFERENCES Roles(roleID)
);

CREATE TABLE Requests (
    requestID SERIAL PRIMARY KEY,
    startDate DATE NOT NULL,
    climateTechType VARCHAR(100) NOT NULL,
    climateTechModel VARCHAR(100) NOT NULL,
    problemDescription TEXT NOT NULL,
    statusID INTEGER NOT NULL,
    completionDate DATE,
    repairParts TEXT,
    masterID INTEGER,
    clientID INTEGER NOT NULL,
    FOREIGN KEY (statusID) REFERENCES Statuses(statusID),
    FOREIGN KEY (masterID) REFERENCES Users(userID),
    FOREIGN KEY (clientID) REFERENCES Users(userID)
);

CREATE TABLE Comments (
    commentID SERIAL PRIMARY KEY,
    message TEXT NOT NULL,
    masterID INTEGER NOT NULL,
    requestID INTEGER NOT NULL,
    FOREIGN KEY (masterID) REFERENCES Users(userID),
    FOREIGN KEY (requestID) REFERENCES Requests(requestID)
);
```

```

INSERT INTO Roles (roleName) VALUES ('Менеджер'),
('Специалист'), ('Оператор'), ('Заказчик');
INSERT INTO Statuses (statusName) VALUES ('В процессе ремонта'),
('Готова к выдаче'), ('Новая заявка');

```

```

INSERT INTO Users (userID, fio, phone, login, password, roleID)
VALUES
(1, 'Широков Василий Матвеевич', '89210563128', 'login1',
'pass1', 1),
(2, 'Кудрявцева Ева Ивановна', '89535078985', 'login2', 'pass2',
2),
(3, 'Гончарова Ульяна Ярославовна', '89210673849', 'login3',
'pass3', 2),
(4, 'Гусева Виктория Данииловна', '89990563748', 'login4',
'pass4', 3),
(5, 'Баранов Артём Юрьевич', '89994563847', 'login5', 'pass5',
3),
(6, 'Овчинников Фёдор Никитич', '89219567849', 'login6',
'pass6', 4),
(7, 'Петров Никита Артёмович', '89219567841', 'login7',
'pass7', 4),
(8, 'Ковалева Софья Владимировна', '89219567842', 'login8',
'pass8', 4),
(9, 'Кузнецов Сергей Матвеевич', '89219567843', 'login9',
'pass9', 4),
(10, 'Беспалова Екатерина Даниэльевна', '89219567844',
'login10', 'pass10', 2);

```

```

INSERT INTO Requests (requestID, startDate, climateTechType,
climateTechModel, problemDescription, statusID, completionDate,
repairParts, masterID, clientID) VALUES
(1, '2023-06-06', 'Кондиционер', 'TCL TAC-12CHSA/TPG-W белый',
'Не охлаждает воздух', 1, NULL, NULL, 2, 7),
(2, '2023-05-05', 'Кондиционер', 'Electrolux EACS/I-
09HAT/N3_21Y белый', 'Выключается сам по себе', 1, NULL, NULL, 3, 8),
(3, '2022-07-07', 'Увлажнитель воздуха', 'Xiaomi Smart
Humidifier 2', 'Пар имеет неприятный запах', 2, '2023-01-01', NULL,
3, 9),
(4, '2023-08-02', 'Увлажнитель воздуха', 'Polaris PUN 2300 WIFI
IQ Home', 'Продолжает работать при снижении воды', 3, NULL, NULL, NULL,
8),
(5, '2023-08-02', 'Сушилка для рук', 'Ballu BAHN-1250', 'Не
работает', 3, NULL, NULL, NULL, 9);

```

```

INSERT INTO Comments (commentID, message, masterID, requestID)
VALUES
(1, 'Всё сделаем!', 2, 1),
(2, 'Всё сделаем!', 3, 2),
(3, 'Починим в момент.', 3, 3);

```

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		48

Приложение Б
(справочная)
Листинг программы

Файл Program.cs

```
public class WebApp
{
    public static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);

        builder.Services.AddRazorPages(); // регистрация Razor
Pages для работы с веб-страницами

        // подключение к базе данных PostgreSQL

builder.Services.AddDbContext<ApplicationDbContext>(options =>

options.UseNpgsql(builder.Configuration.GetConnectionString("Default
Connection")));

        // настройка авторизации

builder.Services.AddAuthentication(CookieAuthenticationDefaults.Auth
enticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Login";
        options.LogoutPath = "/Logout";
        options.AccessDeniedPath = "/Login";
        options.ExpireTimeSpan = TimeSpan.FromHours(8);
    });

builder.Services.AddAuthorization();
var app = builder.Build();

// настройка конвейера HTTP-запросов
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
    app.UseHsts();
}
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapRazorPages();
app.Run();
    }
}
```

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		49

Файл ApplicationDbContext.cs

```

using Microsoft.EntityFrameworkCore;
using Program.Models;

namespace Program.Data;

public class ApplicationDbContext : DbContext
{
    public
ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {

        public DbSet<Role> Roles { get; set; }
        public DbSet<User> Users { get; set; }
        public DbSet<Product> Products { get; set; }
        public DbSet<Service> Services { get; set; }
        public DbSet<Sale> Sales { get; set; }
        public DbSet<SaleItem> SaleItems { get; set; }
        public DbSet<ServiceOrder> ServiceOrders { get; set; }
        public DbSet<ServiceOrderItem> ServiceOrderItems { get; set; }
    }

    protected override void OnModelCreating(ModelBuilder
modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // настройка таблицы Roles
        modelBuilder.Entity<Role>(entity =>
        {
            entity.ToTable("roles");
            entity.HasKey(e => e.RoleId);
            entity.Property(e                               =>
e.RoleId).HasColumnName("role_id");
            entity.Property(e                               =>
e.RoleName).HasColumnName("role_name").HasMaxLength(50).IsRequired()
;
            entity.HasIndex(e => e.RoleName).IsUnique();
        });

        // настройка таблицы Users
        modelBuilder.Entity<User>(entity =>
        {
            entity.ToTable("users");
            entity.HasKey(e => e.UserId);
            entity.Property(e                               =>
e.UserId).HasColumnName("user_id");
            entity.Property(e                               =>
e.Login).HasColumnName("login").HasMaxLength(50).IsRequired();

```

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		50

```

        entity.Property(e
e.PasswordHash).HasColumnName("password_hash");
        entity.Property(e
e.FullName).HasColumnName("full_name").HasMaxLength(100).IsRequired(
);
        entity.Property(e
e.Phone).HasColumnName("phone").HasMaxLength(20);
        entity.Property(e
e.RoleId).HasColumnName("role_id");
        entity.Property(e
e.IsActive).HasColumnName("is_active");
        entity.HasIndex(e => e.Login).IsUnique();
        entity.HasOne(e => e.Role)
            .WithMany(r => r.Users)
            .HasForeignKey(e => e.RoleId)
            .OnDelete(DeleteBehavior.Restrict);
    });

    // настройка таблицы Products
    modelBuilder.Entity<Product>(entity =>
    {
        entity.ToTable("products");
        entity.HasKey(e => e.ProductId);
        entity.Property(e
e.ProductId).HasColumnName("product_id");
        entity.Property(e
e.Name).HasColumnName("name").HasMaxLength(200).IsRequired();
        entity.Property(e
e.Description).HasColumnName("description");
        entity.Property(e
e.Price).HasColumnName("price").HasColumnType("decimal(10,2)").IsRequired();
        entity.Property(e
e.StockQuantity).HasColumnName("stock_quantity");
        entity.Property(e
e.IsActive).HasColumnName("is_active");
    });

    // настройка таблицы Services
    modelBuilder.Entity<Service>(entity =>
    {
        entity.ToTable("services");
        entity.HasKey(e => e.ServiceId);
        entity.Property(e
e.ServiceId).HasColumnName("service_id");
        entity.Property(e
e.Name).HasColumnName("name").HasMaxLength(200).IsRequired();
        entity.Property(e
e.Description).HasColumnName("description");
        entity.Property(e
e.Price).HasColumnName("price").HasColumnType("decimal(10,2)").IsRequired();
    });

```

```

        entity.Property(e
e.IsActive).HasColumnName("is_active");
    });

    // настройка таблицы Sales
    modelBuilder.Entity<Sale>(entity =>
    {
        entity.ToTable("sales");
        entity.HasKey(e => e.SaleId);
        entity.Property(e
e.SaleId).HasColumnName("sale_id");
        entity.Property(e
e.SellerId).HasColumnName("seller_id");
        entity.Property(e
e.SaleDate).HasColumnName("sale_date");
        entity.Property(e
e.TotalAmount).HasColumnName("total_amount").HasColumnType("decimal(
10,2)");

        entity.HasOne(e => e.Seller)
            .WithMany()
            .HasForeignKey(e => e.SellerId)
            .OnDelete(DeleteBehavior.Restrict);
    });

    // настройка таблицы SaleItems
    modelBuilder.Entity<SaleItem>(entity =>
    {
        entity.ToTable("saleitems");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Id).HasColumnName("id");
        entity.Property(e
e.SaleId).HasColumnName("sale_id");
        entity.Property(e
e.ProductId).HasColumnName("product_id");
        entity.Property(e
e.Quantity).HasColumnName("quantity");
        entity.Property(e
e.PriceAtSale).HasColumnName("price_at_sale").HasColumnType("decimal
(10,2)");

        entity.HasOne(e => e.Sale)
            .WithMany(s => s.SaleItems)
            .HasForeignKey(e => e.SaleId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(e => e.Product)
            .WithMany()
            .HasForeignKey(e => e.ProductId)
            .OnDelete(DeleteBehavior.Restrict);
    });

    // настройка таблицы ServiceOrders
    modelBuilder.Entity<ServiceOrder>(entity =>
    {
        entity.ToTable("serviceorders");

```

```

        entity.HasKey(e => e.OrderId);
        entity.Property(e => e.OrderId).HasColumnName("order_id");
        entity.Property(e => e.MasterId).HasColumnName("master_id");
        entity.Property(e => e.OrderDate).HasColumnName("order_date");
        entity.Property(e => e.TotalAmount).HasColumnName("total_amount").HasColumnType("decimal(10,2)");
        entity.Property(e => e.Notes).HasColumnName("notes");
        entity.HasOne(e => e.Master)
            .WithMany()
            .HasForeignKey(e => e.MasterId)
            .OnDelete(DeleteBehavior.Restrict);
    });

    // настройка таблицы ServiceOrderItems
    modelBuilder.Entity<ServiceOrderItem>(entity =>
    {
        entity.ToTable("serviceorderitems");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Id).HasColumnName("id");
        entity.Property(e => e.OrderId).HasColumnName("order_id");
        entity.Property(e => e.ServiceId).HasColumnName("service_id");
        entity.Property(e => e.PriceAtOrder).HasColumnName("price_at_order").HasColumnType("decimal(10,2)");
        entity.HasOne(e => e.ServiceOrder)
            .WithMany(so => so.ServiceOrderItems)
            .HasForeignKey(e => e.OrderId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(e => e.Service)
            .WithMany()
            .HasForeignKey(e => e.ServiceId)
            .OnDelete(DeleteBehavior.Restrict);
    });
}

```

Файл Service.cs

```

namespace Program.Models;

public class Service
{
    public int ServiceId { get; set; }
    public string Name { get; set; } = string.Empty;
    public string? Description { get; set; }
}

```

```

        public decimal Price { get; set; }
        public bool IsActive { get; set; } = true;
    }

```

Файл Login.cshtml.cs

```

[AllowAnonymous]
public class LoginModel : PageModel
{
    private readonly ApplicationDbContext _context;

    [BindProperty]
    public string Login { get; set; } = string.Empty;

    [BindProperty]
    public string Password { get; set; } = string.Empty;

    public async Task<IActionResult> OnPostAsync()
    {
        // Проверка наличия логина и пароля
        if (string.IsNullOrEmpty(Login) ||
            string.IsNullOrEmpty>Password))
        {
            ErrorMessage = "Введите логин и пароль";
            return Page();
        }

        // Поиск пользователя в базе данных
        var user = await _context.Users
            .Include(u => u.Role)
            .FirstOrDefaultAsync(u => u.Login == Login &&
u.IsActive);

        if (user == null)
        {
            ErrorMessage = "Неверный логин или пароль";
            return Page();
        }

        // Проверка пароля (поддержка BCrypt и открытого текста)
        bool isPasswordValid = false;
        if (user.PasswordHash.StartsWith("$2a$") ||
            user.PasswordHash.StartsWith("$2b$"))
        {
            isPasswordValid =
BCrypt.Net.BCrypt.Verify>Password, user.PasswordHash);
        }
        else
        {
            isPasswordValid = user.PasswordHash == Password;
        }
        if (!isPasswordValid)

```

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		54

```

        {
            ErrorMessage = "Неверный логин или пароль";
            return Page();
        }

        // Создание claims для аутентификации
        var claims = new List<Claim>
        {
            new Claim(ClaimTypes.NameIdentifier,
user.UserId.ToString()),
            new Claim(ClaimTypes.Name, user.FullName),
            new Claim(ClaimTypes.Role, user.Role?.RoleName ??
""),
            new Claim("Login", user.Login)
        };

        var claimsIdentity = new ClaimsIdentity(claims,
CookieAuthenticationDefaults.AuthenticationScheme);
        var authProperties = new AuthenticationProperties
        {
            IsPersistent = true,
            ExpiresUtc = DateTimeOffset.UtcNow.AddHours(8)
        };

        // Вход в систему
        await HttpContext.SignInAsync(
            CookieAuthenticationDefaults.AuthenticationScheme,
            new ClaimsPrincipal(claimsIdentity),
            authProperties);

        return RedirectToPage("/Dashboard");
    }
}

```

Файл Main.cshtml.cs

```

// загрузка данных
private async Task LoadDataAsync()
{
    var query = SearchQuery?.ToLower() ?? string.Empty;

    switch (ActiveTab.ToLower())
    {
        case "products":
            var productsQuery =
_context.Products.AsQueryable();
            if (!string.IsNullOrEmpty(query))
            {
                productsQuery = productsQuery.Where(p =>
                    p.Name.ToLower().Contains(query) ||
                    (p.Description != null &&
p.Description.ToLower().Contains(query)));
            }
        }
    }
}

```

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		55

```

        }
        if (FilterActive == "active")
        {
            productsQuery = productsQuery.Where(p =>
p.IsActive);
        }
        Products = await productsQuery.OrderBy(p =>
p.Name).ToListAsync();
        break;
        // ... аналогично для других вкладок
    }
}

// создание продажи

public async Task<IActionResult> OnPostCreateSale()
{
    // получение ID текущего пользователя
    var userIdClaim =
User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
    if (!int.TryParse(userIdClaim, out var sellerId) || sellerId
<= 0)
    {
        return RedirectToPage("/Dashboard", new { tab = "sales"
});
    }

    // парсинг данных из формы
    var form = Request.Form;
    var indices = new List<int>();
    // ... получение индексов товаров из формы

    // создание списка товаров
    SaleItems = new List<SaleItemData>();
    foreach (var i in indices)
    {
        // парсинг данных о товаре
        SaleItems.Add(new SaleItemData { ... });
    }

    // использование транзакции для атомарности
    using var transaction = await
_context.Database.BeginTransactionAsync();

    try
    {
        // проверка наличия товаров на складе
        foreach (var itemData in validItems)
        {
            var product = await
_context.Products.FindAsync(itemData.ProductId);
            if (product == null || !product.IsActive)

```

					09.02.07.000022 П. 515	Лист
Изм.	Лист	№ докум.	Подпись	Дата		56


```

        {
            await transaction.RollbackAsync();
            return RedirectToPage("/Dashboard", new { tab =
"sales" });
        }
        if (product.StockQuantity < itemData.Quantity)
        {
            await transaction.RollbackAsync();
            return RedirectToPage("/Dashboard", new { tab =
"sales" });
        }
    }

    // создание продажи
    var sale = new Models.Sale
    {
        SellerId = sellerId,
        SaleDate = DateTime.UtcNow,
        TotalAmount = 0 // Рассчитывается триггером в БД
    };
    _context.Sales.Add(sale);
    await _context.SaveChangesAsync();

    // добавление позиций продажи
    foreach (var itemData in validItems)
    {
        var saleItem = new Models.SaleItem
        {
            SaleId = sale.SaleId,
            ProductId = itemData.ProductId,
            Quantity = itemData.Quantity,
            PriceAtSale = itemData.PriceAtSale
        };
        _context.SaleItems.Add(saleItem);
    }

    await _context.SaveChangesAsync();
    await transaction.CommitAsync();

    return RedirectToPage("/Dashboard", new { tab = "sales"
});
    }
    catch
    {
        await transaction.RollbackAsync();
        throw;
    }
}

// сохранение товара

```

```

        public async Task<IActionResult> OnPostSaveProduct(int?
ProductId, string Name,
        string? Description, decimal Price, int StockQuantity,
string? IsActive)
        {
            if (string.IsNullOrEmpty(Name) || Price < 0)
            {
                return RedirectToPage("/Dashboard", new { tab =
"products" });
            }

            if (ProductId.HasValue && ProductId.Value > 0)
            {
                // редактирование существующего товара
                var product = await
_context.Products.FindAsync(ProductId.Value);
                if (product != null)
                {
                    product.Name = Name;
                    product.Description = Description;
                    product.Price = Price;
                    product.StockQuantity = StockQuantity;
                    product.IsActive = IsActive == "true" || IsActive ==
"True";

                    await _context.SaveChangesAsync();
                }
            }
            else
            {
                // создание нового товара
                var product = new Models.Product
                {
                    Name = Name,
                    Description = Description,
                    Price = Price,
                    StockQuantity = StockQuantity,
                    IsActive = IsActive == "true" || IsActive == "True"
                };
                _context.Products.Add(product);
                await _context.SaveChangesAsync();
            }

            return RedirectToPage("/Dashboard", new { tab = "products"
});
        }

```